ProcurePulse: Final Report

Gold Price Direction, Buy/Hold/Negotiate Recommendations and Application

Hassan Ashraf 24L-3073

Sharjeel Shahid 24L-3055

November 2026

Abstract

Small and medium-sized businesses that buy metals pay prices set on commodity exchanges, and they rarely have the tools to judge whether a supplier's quote is fair or whether to buy now or wait. ProcurePulse addresses this problem. We collected daily data from January 2010 to October 2026 for four COMEX metals (gold, silver, copper and platinum) and four economic signals (the US dollar index, crude oil, the 10-year Treasury yield and inflation), cleaned it and built 27 features. We then predicted whether the gold price will be higher in five trading days, comparing a majority baseline, logistic regression, a random forest, histogram gradient boosting and an LSTM network. The models were tuned with walk-forward validation on 2010–2021, selected on 2022–2023 and tested once on 701 trading days from 2024 to October 2026. The signal is weak: the best test AUC-ROC is 0.60, logistic regression gives the most reliable "up" calls (precision 0.70) and the LSTM gives the most balanced predictions (F1 0.55), while the tree models predict "down" on every test day because of a shift in price levels. We turned these results into Buy, Hold and Negotiate recommendations: each quote is compared with an expected five-day price range built from the 10th, 50th and 90th percentiles of past five-day price changes, and for gold the model's predicted direction decides quotes inside that range. Finally, we built a local Streamlit application that runs this inference in real time on built-in, uploaded or freshly downloaded prices and explains every recommendation.

Contents

1 Introduction **3**

2 Problem Definition **3**

2.1 Problem Statement 3

2.2 Target Audience 5

2.3 AI Justification 5

3 Data Collection **6**

3.1 Data Sources 6

3.2 Collection Method 6

3.3 Raw Data Description 8

3.4 Data Quality Issues Found 9

4 Data Cleaning **10**

4.1 Overview 10

4.2 Cleaning Pipeline Summary 10

4.3 Methodological Justifications 11

5 Feature Strategy and Exploratory Data Analysis **12**
5.1 Feature Engineering Strategy . . . 12
5.2 Calendar Encoding and Categoricals . . . 12
5.3 Chronological Dataset Partitioning . . . 13
5.4 Outlier Capping and Feature Scaling . . . 13
5.5 Exploratory Data Analysis . . . 13

6 Classical Models (Scikit-Learn) **19**
6.1 Task and Framework Choice . . . 19
6.2 Models . . . 19
6.3 Hyperparameter Tuning . . . 19
6.4 Final Choice . . . 20
6.5 Feature Importance . . . 21

7 Deep Learning Model (PyTorch) **21**
7.1 Framework and Motivation . . . 21
7.2 Input Windows . . . 22
7.3 Architecture . . . 22
7.4 Loss and Optimizer . . . 23
7.5 Hyperparameter Tuning . . . 23
7.6 Training Behaviour . . . 24
7.7 Result Summary . . . 26

8 Evaluation **26**
8.1 Validation Strategy . . . 26
8.2 Metric Selection . . . 26
8.3 Results . . . 27
8.4 Discussion . . . 28

9 Error Analysis and Bias **29**
9.1 Where the Models Fail . . . 29
9.2 Bias Check . . . 32

10 Buy, Hold and Negotiate Recommendations **34**
10.1 Design Choice: AI Model for Gold, Price Range for All Metals . . . 34
10.2 Expected Five-Day Price Range . . . 34
10.3 Decision Rules . . . 35
10.4 Worked Example . . . 35
10.5 Fresh Data . . . 35
10.6 Limitations . . . 36

11 Application (Track A) **36**
11.1 Why Track A and Streamlit . . . 36
11.2 Application Architecture . . . 36
11.3 Real-Time Inference Flow . . . 37
11.4 Data Options on Quote Check . . . 38
11.5 Screenshots . . . 38
11.6 Sample Predictions . . . 40
11.7 Running the Application and Limitations . . . 41

12 System Architecture and Contributions **42**
12.1 System Architecture 42
12.2 Team 42
12.3 Contribution Table: Deliverable 3 43
12.4 Contribution Table: Deliverable 2 44

13 Conclusion **45**
13.1 Future Work 46

1 Introduction

ProcurePulse is an AI-assisted tool that helps businesses decide *when* to buy metals and whether a supplier's price is fair. It studies the prices of gold and other metals traded on COMEX, predicts the short-term direction of gold, and compares a supplier's quote with the expected price range to recommend **Buy**, **Hold** or **Negotiate**. The project also tests whether external economic signals, such as the US dollar, interest rates, oil and inflation, help to predict gold.

This is the final report of the project. It brings together the three deliverables, whose objectives were to:

1. define the real-world problem, the target users and why machine learning is needed (Phase 1);
2. collect 16 years of daily market data from public sources with reproducible, logged Python scripts, then clean it, engineer features and explore it visually (Phase 2);
3. build, tune and compare classical machine learning models and a deep learning model for the five-day direction of gold (Phase 3);
4. evaluate the models with robust validation and suitable metrics, and analyse their errors and bias (Phase 4);
5. turn the results into Buy, Hold and Negotiate recommendations and deliver them through an application with real-time inference (Phase 5, Track A);
6. document the whole project in one report (Phase 6).

The report is organised as follows. Section 2 defines the problem, the target audience and the AI justification. Section 3 describes the data sources and the raw data, Section 4 explains how the data was cleaned, and Section 5 presents the features and the exploratory data analysis. Sections 6 and 7 describe the classical models and the LSTM, Section 8 compares all models on the test years, and Section 9 analyses their errors and bias. Section 10 explains how quotes are turned into recommendations, and Section 11 presents the application. Section 12 shows the system architecture and who built each part, and Section 13 summarises the results and future work. All code is in the project's GitHub repository.

2 Problem Definition

2.1 Problem Statement

Businesses that make products from metals, such as jewellers, electronics assemblers and metal fabricators, buy their raw materials at prices they do not control. These prices are set on commodity

exchanges such as COMEX and change every trading day. Our own data (Figure 1) shows how large these changes are: gold futures traded at about $1,050 per ounce in late 2015 and rose above $5,000 per ounce in early 2026, while copper roughly tripled from its 2016 low. Movements over a single day can also be severe: the largest one-day falls in our 2010–2026 data are −10.8% for gold, −22.3% for copper and −31.3% for silver.

For a buyer, this volatility creates two practical problems:

- **Timing.** Most small and medium-sized businesses buy when stock runs low or on a fixed schedule, at whatever the price is that day. Buying a few weeks earlier or later can change the cost of the same order by several percent, which can be the whole profit margin.
- **Judging a supplier's quote.** Suppliers quote the market price plus a markup. Without a view of where the price is likely to go, the buyer cannot tell whether a quote is fair, generous or inflated.

Large firms handle this with in-house analysts and hedging desks. Smaller firms usually rely on intuition, news headlines or the supplier's own advice, which is not neutral.

ProcurePulse addresses this gap. It forecasts gold and other metal prices from COMEX data, gives each forecast a *prediction interval* (a range the price is expected to fall within, with a stated probability), and compares a supplier's quote against that forecast to recommend one of three actions: **Buy** (the quote is good relative to the expected price), **Hold** (prices are expected to fall, so waiting is likely cheaper) or **Negotiate** (the quote is high relative to the forecast range).

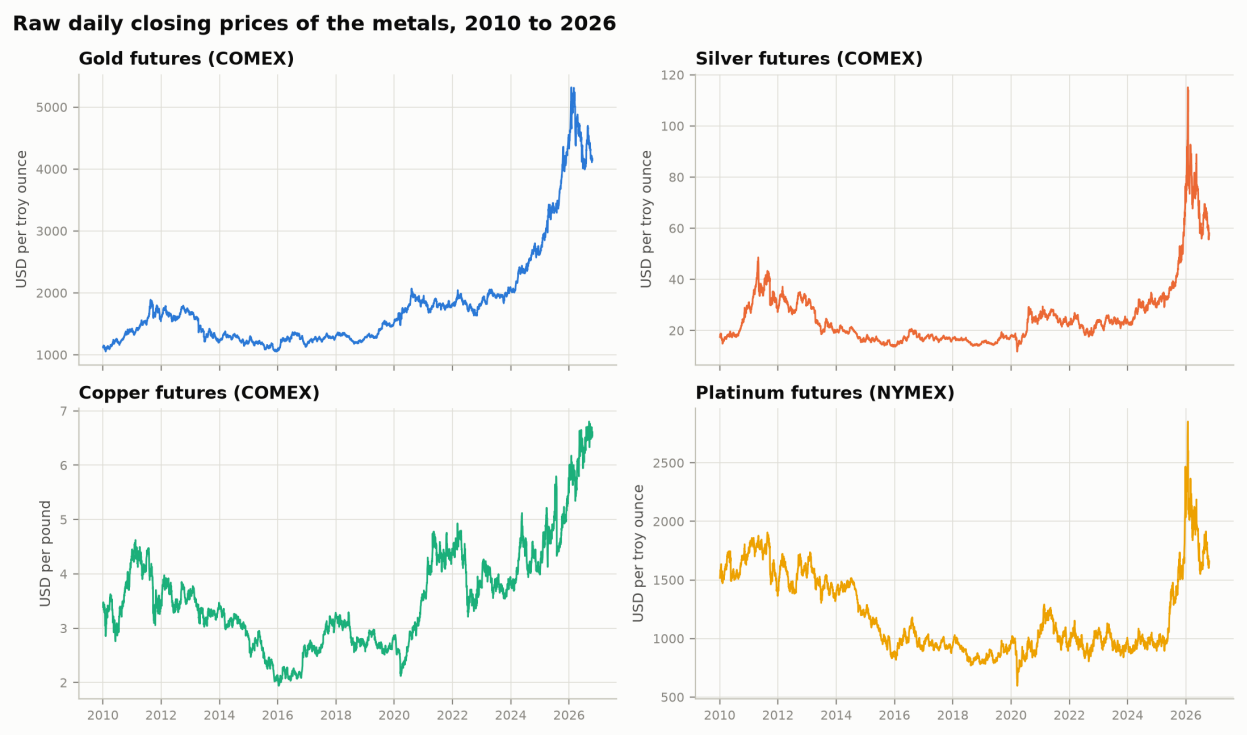


Figure 1: Daily closing prices of COMEX gold, silver and copper futures and NYMEX platinum futures, January 2010 to October 2026 (raw data).

2.2 Target Audience

The primary users are **procurement managers and owners of small and medium-sized enterprises (SMEs) that buy metals regularly** but do not have their own market analysts:

- jewellery manufacturers and gold traders (gold, silver, platinum);
- electronics and electrical-goods assemblers (copper, gold, silver);
- metal fabricators and workshops (copper).

Example user (hypothetical). Sara manages purchasing for a 40-person jewellery workshop that buys about 2 kg of gold each month. Each week she receives quotes from two suppliers and must decide whether to accept one, wait or push for a lower price. She has no time to follow the markets in detail. ProcurePulse gives her a forecast with a clear range and a one-word recommendation she can act on.

A secondary audience is students and analysts who want a transparent, reproducible comparison of forecasting models for commodity prices, including whether external economic signals improve accuracy.

2.3 AI Justification

A rule-based system would encode fixed instructions such as "buy when the price falls 5% below its 30-day average". Such rules fail for this problem for four reasons:

1. **The patterns are not fixed.** Metal prices move through different regimes: long calm periods, sharp trends and sudden shocks (for example March 2020 and early 2026 in Figure 1). A threshold that works in one regime fails in the next, and someone would have to keep rewriting the rules by hand.
2. **Many signals interact.** Gold tends to move against the US dollar and real interest rates, while oil prices and inflation also play a role. The strength of these relationships changes over time and is non-linear, which is very hard to express as hand-written rules.
3. **Uncertainty must be measured, not guessed.** The Buy/Hold/Negotiate decision depends on how likely the price is to move, not only on a single predicted value. Statistical and machine learning models can learn this uncertainty from historical data and express it as a prediction interval; a fixed rule cannot.
4. **The data is large and keeps growing.** More than 4,000 trading days across eight series are far too many for a person to turn into reliable rules, while a model can be retrained automatically as new data arrives.

We therefore treat price forecasting as a **supervised time-series learning problem**. In later deliverables we will compare models of increasing complexity on the same chronological train/validation/test split:

- a **naive baseline** (tomorrow's price equals today's) and **ARIMA**, a classical statistical model [3];
- **Prophet** [22] and the gradient-boosted tree model **XGBoost** [5];

- a **Long Short-Term Memory (LSTM)** recurrent neural network [10], which can learn longer-range patterns in sequences.

Each model will be evaluated with standard forecasting error measures and the coverage of its prediction intervals [11], with and without the external signals, to test whether those signals actually improve accuracy. The approach is only worth using if the learned models beat the naive baseline, and this comparison is what will show it.

3 Data Collection

3.1 Data Sources

We collected daily data from 1 January 2010 to 20 October 2026 for four metals and four external signals (Table 1). All sources are public and free. Metal prices are *continuous front-month futures* prices, which is the benchmark that physical metal suppliers price against.

Column	Description	Source (ID)	Frequency	Rows
`gold`	COMEX gold futures, USD/oz	Yahoo Finance (`GC=F`)	Daily	4,225
`silver`	COMEX silver futures, USD/oz	Yahoo Finance (`SI=F`)	Daily	4,224
`copper`	COMEX copper futures, USD/lb	Yahoo Finance (`HG=F`)	Daily	4,225
`platinum`	NYMEX platinum futures, USD/oz	Yahoo Finance (`PL=F`)	Daily	4,223
`dxy`	US Dollar Index	Yahoo Finance (`DX-Y.NYB`)	Daily	4,226
`wti`	WTI crude oil futures, USD/bbl	Yahoo Finance (`CL=F`)	Daily	4,225
`us10y`	10-year US Treasury yield, %	Yahoo Finance (`^TNX`)	Daily	4,223
`cpi`	US Consumer Price Index (SA)	BLS (`CUSR0000SA0`)	Monthly	201

Table 1: Data sources. Rows are the number of observations downloaded between January 2010 and October 2026.

Why these external signals. Each one is a widely cited driver of metal prices, and testing whether they improve forecasts is part of our project:

- **US Dollar Index:** metals are priced in US dollars, so a stronger dollar makes them more expensive for other buyers and tends to push prices down.
- **10-year Treasury yield:** gold pays no interest, so when interest rates rise, holding gold becomes less attractive compared with bonds.
- **WTI crude oil:** oil feeds into mining and transport costs and into general inflation.
- **Consumer Price Index:** gold is often bought as a hedge against inflation, which the CPI measures.

3.2 Collection Method

Collection is done by three Python scripts in `data_scripts/`, run in this order:

1. `fetch_metals.py` downloads the four metal futures from Yahoo Finance using the `yfinance` library [1] and saves one CSV file per metal.
2. `fetch_external.py` downloads the dollar index and oil from Yahoo Finance, and the 10-year yield and CPI from FRED [8].

3. `merge_sources.py` joins all eight series on their date into one table, `merged_raw.csv`.

Every download is retried up to three times, and each step writes a line to `data_scripts/logs/collection.log` with the number of rows, the date range and the number of missing values. The log is our evidence of collection; a copy is included in the repository as `report/evidence/collection_log.txt`. Raw data files are not uploaded to GitHub; anyone can recreate them by running the three scripts. All shared settings (tickers, start date and file paths) are kept in one file, `config.py`.

Backup sources. During collection, FRED repeatedly refused or timed out the connection from our network. Instead of failing, `fetch_external.py` automatically switched to backup sources, and recorded the switch in the log:

- the 10-year yield was taken from Yahoo Finance's `^TNX` (the CBOE 10-year Treasury yield index, quoted in percent), which tracks FRED's `DGS10` series closely;
- CPI was taken directly from the US Bureau of Labor Statistics API [23], series `CUSR0000SA0`. This is the same seasonally adjusted CPI that FRED republishes as `CPIAUCSL`. The free API returns at most ten years per request, so the script requests 2010–2019 and 2020–2026 separately.

The output of the collection scripts is shown below and in Figure 2.

```
gold (GC=F): 4225 rows, 2010-01-04 to 2026-10-20, 0 missing
silver (SI=F): 4224 rows, 2010-01-04 to 2026-10-20, 0 missing
copper (HG=F): 4225 rows, 2010-01-04 to 2026-10-20, 0 missing
platinum (PL=F): 4223 rows, 2010-01-04 to 2026-10-20, 0 missing
dxy (DX-Y.NYB): 4226 rows, 2010-01-04 to 2026-10-20, 0 missing
wti (CL=F): 4225 rows, 2010-01-04 to 2026-10-20, 0 missing
us10y: FRED unavailable, using backup source...
us10y (Yahoo ^TNX): 4223 rows, 2010-01-04 to 2026-10-20, 0 missing
cpi: FRED unavailable, using backup source...
cpi (BLS CUSR0000SA0): 201 rows, 2010-01-01 to 2026-09-01, 1 missing
merged_raw.csv: 4300 rows x 8 columns, 2010-01-01 to 2026-10-20
```

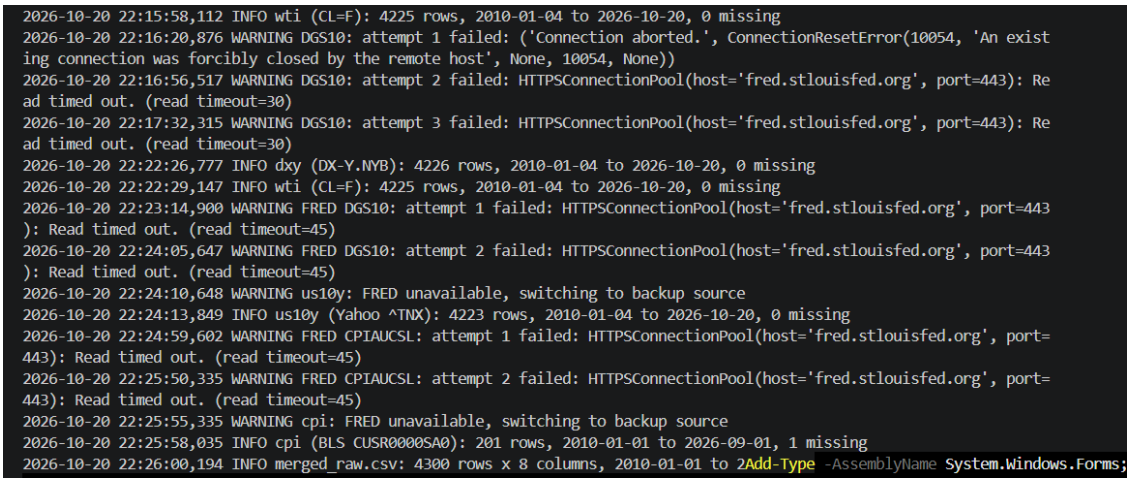


Figure 2: Running the collection scripts in the terminal.

3.3 Raw Data Description

The merged raw table has **4,300 rows and 8 columns**, one row for every date that appears in at least one source, from 1 January 2010 to 20 October 2026. Gaps are deliberately left empty at this stage; they are handled in cleaning (Section 4). Figures 1 and 3 show the raw series, and Table 2 and Figure 4 summarise the missing values.

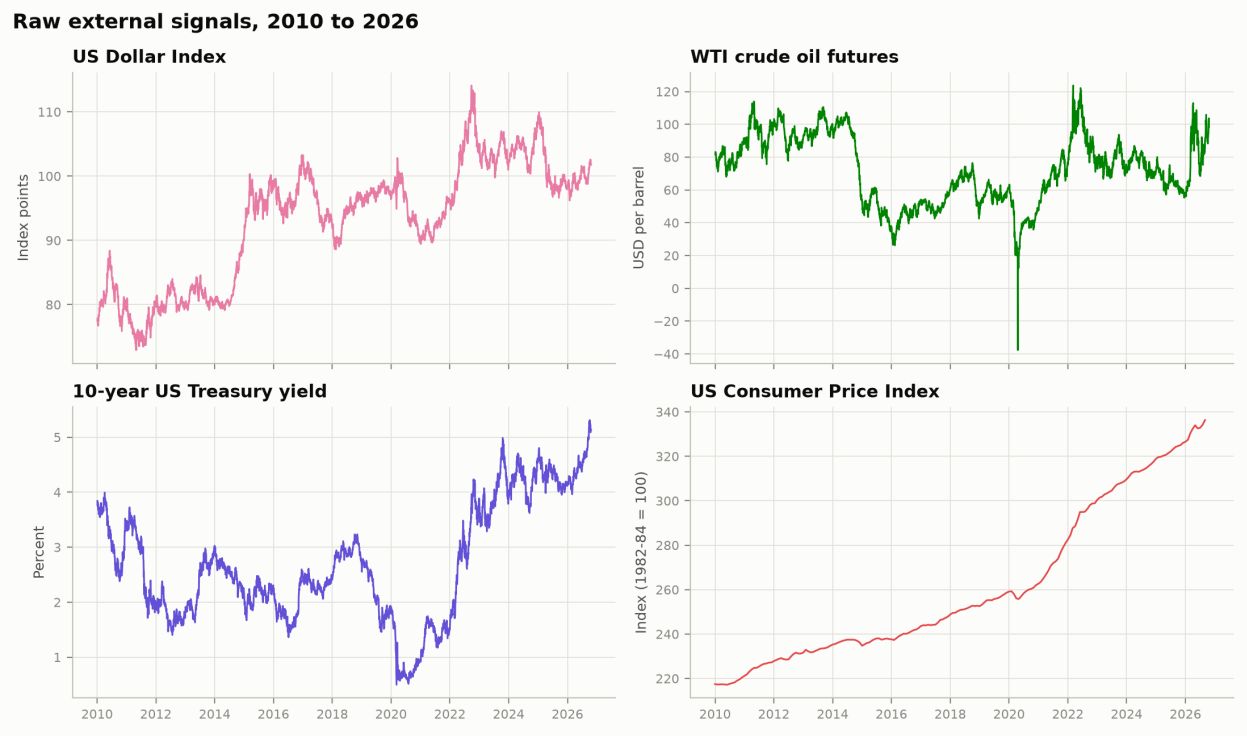


Figure 3: Raw external signals, 2010–2026. Note the WTI oil futures price falling below zero in April 2020.

Column	Missing rows	Share of 4,300 rows
`gold`	75	1.7%
`silver`	76	1.8%
`copper`	75	1.7%
`platinum`	77	1.8%
`dxy`	74	1.7%
`wti`	75	1.7%
`us10y`	77	1.8%
`cpi`	4,100	95.3%

Table 2: Missing values per column in `merged_raw.csv`.

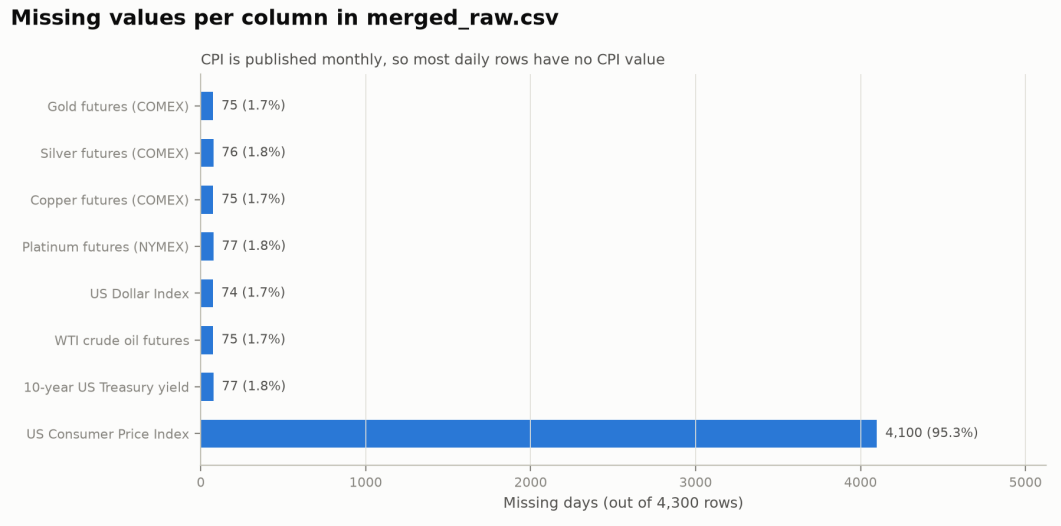


Figure 4: Missing values per column in the raw merged data.

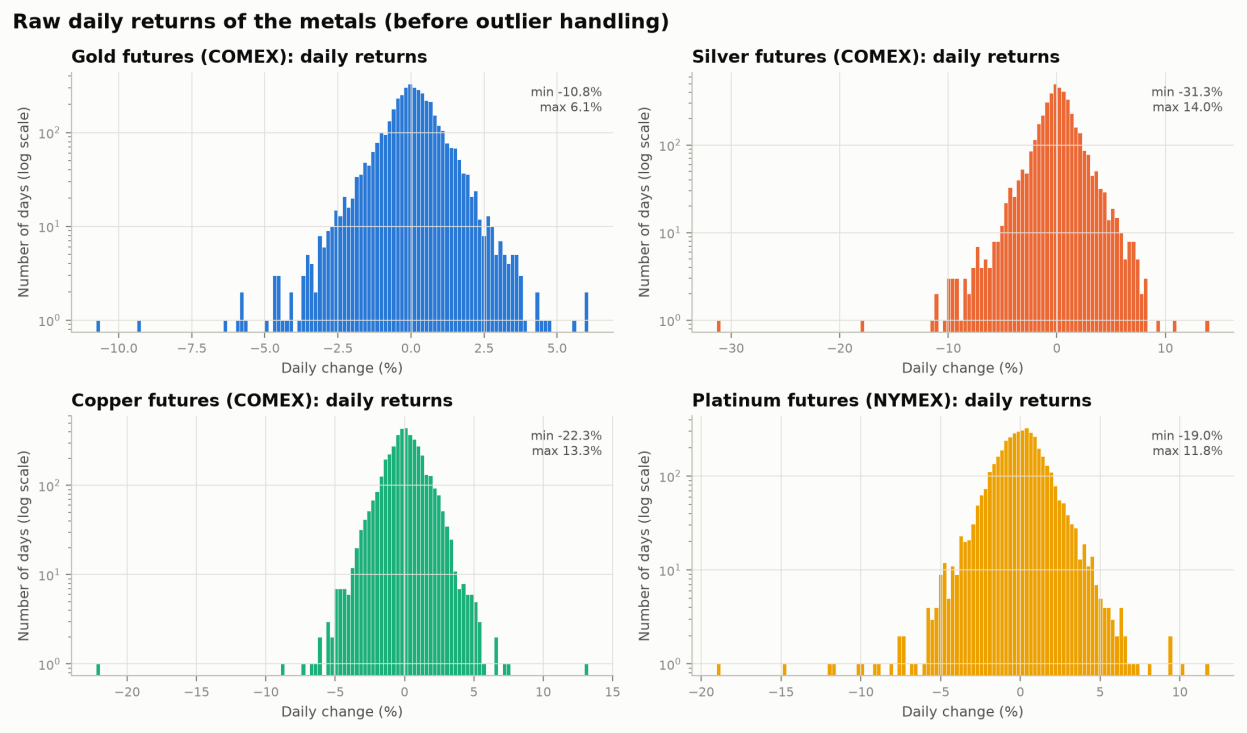


Figure 5: Distribution of raw daily returns of the four metals (log scale). The long tails show rare extreme days.

3.4 Data Quality Issues Found

Inspecting the raw data revealed the following issues, which are handled in Section 4:

- **Gaps from different calendars.** About 75 rows (1.7–1.8%) are missing in each daily series because markets close on different holidays, so a date present in one source can be absent in

another.

- **Mixed frequencies.** CPI is published monthly, so 95% of the daily rows have no CPI value. One month is also missing in the BLS series itself.
- **An impossible price.** WTI oil futures closed at about −$37 per barrel on 20 April 2020 (Figure 3). This was a real event caused by an expiring contract, but a negative price cannot be used for percentage returns.
- **Extreme daily moves.** The largest one-day falls are −10.8% for gold, −19.0% for platinum, −22.3% for copper and −31.3% for silver (Figure 5). Some are genuine market crashes; others may come from the way a continuous futures series jumps from one contract to the next. They must be detected and treated as outliers.

4 Data Cleaning

4.1 Overview

The raw merged dataset (`merged_raw.csv`) contains 4,300 rows and 8 columns, spanning from 1 January 2010 to 20 October 2026. Because each financial and macroeconomic series originates from independent reporting calendars and publishing frequencies, the merged data contains cross-calendar gaps, monthly reporting intervals, an unphysical negative contract price, and severe return outliers. We developed a sequential, leak-free cleaning pipeline in `data_scripts/clean_data.py` to resolve these issues systematically. All operations, row transitions, and column-level missing values were recorded during execution in our audit trail, preserved in `report/evidence/cleaning_log.txt`.

4.2 Cleaning Pipeline Summary

Table 3 outlines the sequence of cleaning operations, the respective methods applied, and the before-and-after observation counts across the dataset.

Issue	Method	Before	After
Duplicate dates	Keep last observation	0	0
Missing CPI (monthly, dated the 1st)	Forward-fill	4,100	0
Non-trading days (all four metals missing)	Drop rows	4,300	4,225 rows
Short gaps in single series	Time interpolation, limit 3 consecutive trading days, inside only	10	0
Remaining missing	Dropna	0 rows removed	0
Non-positive prices	Set NaN then time-interpolate	1 value (WTI)	0
Outlier days (3 × IQR on daily returns)	Flagged via `is_outlier`, not deleted	129 days flagged	4,225

Table 3: Data cleaning pipeline summary showing the resolution method and effect of each sequential step.

The individual outlier counts detected across the price series are 31 for gold, 57 for silver, 12 for copper, 24 for platinum, 6 for the US Dollar Index (`dxy`), and 45 for WTI crude oil (`wti`). Across the entire series, 129 days are flagged as containing an outlier in at least one asset, representing about 3.0% of the 4,225 trading days.

4.3 Methodological Justifications

Order of CPI forward-filling. Consumer Price Index data is published monthly on the 1st calendar day of each month. Because the 1st of a month frequently coincides with a weekend or public holiday when exchange markets are closed, dropping non-trading days prior to forward-filling would discard the row bearing the newly released CPI figure, permanently losing that inflation update. By forward-filling the CPI column first, the newly published index level is propagated forward onto active subsequent trading dates. This forward-fill strictly projects past, officially published figures into the future and introduces zero lookahead leakage.

Time interpolation for short gaps. To bridge brief isolated exchange holidays where a single series is absent while other markets trade, we apply time-weighted interpolation, up to 3 consecutive missing trading days, inside only. Restricting interpolation to strictly inside boundaries ensures that missing values are only interpolated between known observations, preventing unconstrained extrapolation at the historical edges of the dataset.

Return-based outlier detection. We detect statistical outliers on daily percentage returns rather than raw price levels. Gold futures closed at about \$1,118 per ounce on 4 January 2010 and reached about \$4,134 on 20 October 2026. A fixed threshold applied to raw price levels would flag most recent trading days simply due to long-term nominal price growth. Daily returns are centred near zero and much closer to stationary than prices, providing a stable baseline for statistical dispersion metrics over a sixteen-year horizon.

Three-times interquartile range threshold. We adopt a $3 \times$ IQR boundary ($[Q_1 - 3 \times \text{IQR}, Q_3 + 3 \times \text{IQR}]$) rather than the standard Tukey $1.5 \times$ IQR rule. Empirical financial returns possess heavy tails and volatility clustering; setting a $1.5 \times$ IQR limit would label normal, high-volatility market sessions as anomalies. A $3 \times$IQR threshold reserves outlier labels for extreme tail moves.

Flagging rather than row deletion. We record outlier occurrences using a binary indicator column (`is_outlier`) without removing or capping rows during data cleaning. Deleting rows breaks the contiguous chronological spacing required by autoregressive models and loses valid information from genuine historical market dislocations, such as the market turbulence of March 2020. Flagging preserves full historical variance while allowing downstream models to identify shock sessions explicitly.

Impossible non-positive values. Asset prices cannot be zero or negative in return computations, as division by non-positive bases yields undefined or uninterpretable percentage returns. In our raw dataset, the front-month WTI contract closed at $-\$37.63$ on 20 April 2020. We mask non-positive values in price columns as `NaN` and apply time-weighted interpolation to restore continuous, positive price series.

All executed operations, step-by-step missing counts, and row counts were logged directly to `report/evidence/cleaning_log.txt`.

5 Feature Strategy and Exploratory Data Analysis

5.1 Feature Engineering Strategy

Raw commodity prices and macroeconomic signals cannot be supplied directly to machine learning architectures without transformation. We construct temporal, autoregressive, rolling statistical, and cyclical features in `data_scripts/feature_engineering.py` using `pandas` [14]. All feature transformations use strictly contemporaneous or historical information to ensure no lookahead bias. Table 4 summarizes the engineered features.

Feature Name	Derivation	Predictive Rationale
`ret_<asset>`	Percentage change of asset price	Scale-free daily change across metals and signals
`ret_us10y`	First difference of 10-year Treasury yield	Change in percentage points (yield quoted in %)
`logret_gold`	Logarithmic difference $\ln(P_t) - \ln(P_{t-1})$	Continuous compounded returns for modeling
`gold_lag1/5/20`	Shift gold price by 1, 5, and 20 trading days	Autoregressive persistence over 1 day, 1 week, 1 month
`gold_ma7/ma30`	7-day and 30-day rolling arithmetic mean	Short-term trend momentum and medium-term baseline
`gold_vol30`	30-day rolling standard deviation of `ret_gold`	Dynamic return volatility and risk clustering
`month_sin/cos`	$\sin(2\pi m/12)$ and $\cos(2\pi m/12)$	Continuous cyclical monthly calendar encoding
`dow_sin/cos`	$\sin(2\pi d/5)$ and $\cos(2\pi d/5)$	Continuous cyclical trading day-of-week encoding
`gold_5d_ahead`	Negative shift by 5 trading days	Regression target: expected price 1 trading week ahead
`direction_5d`	Binary flag: (`gold_5d_ahead` > `gold`)	Classification target: 1 if future price rises, else 0

Table 4: Engineered features and forecasting targets. Forecasting targets are strictly excluded from model inputs.

We emphasize that `gold_5d_ahead` and `direction_5d` are forward-looking prediction targets and are never included as model inputs. Constructing the rolling windows and forward targets reduces the dataset from 4,225 to 4,190 rows: 30 leading rows are lost to the 30-day rolling windows and lags, and 5 trailing rows are dropped because they have no 5-day-ahead future price. The resulting dataset contains 29 columns, including the binary `is_outlier` flag.

5.2 Calendar Encoding and Categoricals

We encode calendar variables using cyclical sine and cosine transformations rather than one-hot encoding. One-hot encoding would add $12+5 = 17$ sparse binary columns and treat calendar intervals as mutually independent categories, discarding proximity. In contrast, trigonometric functions map calendar steps onto a continuous two-dimensional circle, preserving proximity between December and January as well as between Friday and Monday across the weekend. Because our dataset is structured in wide tabular time-series format, there are no text categories to one-hot encode, and `is_outlier` is already a native binary indicator.

5.3 Chronological Dataset Partitioning

To evaluate model generalisation in a time-series setting, we partition the dataset strictly chronologically without shuffling. Random partitioning or cross-validation would mix future observations into training folds, causing temporal data leakage. Table 5 details the three non-overlapping chronological partitions.

Split	Date Range	Rows	Share of `direction_5d == 1`
Train	2010-02-17 to 2021-12-31	2,988	54.25%
Validation	2022-01-03 to 2023-12-29	502	51.79%
Test	2024-01-02 to 2026-10-13	700	60.86%

Table 5: Chronological train, validation, and test partitions.

The training split provides nearly twelve years of historical data covering varied market regimes. The validation split serves for hyperparameter tuning and model selection, while the test split provides an out-of-sample holdout spanning 2024 to late 2026.

5.4 Outlier Capping and Feature Scaling

We apply return capping and feature scaling strictly on the training set to prevent data leakage. Data leakage occurs when statistics from validation or test periods inform preprocessing parameters, creating artificially optimistic evaluation metrics.

Return capping. We identify all daily return columns (`ret_*` and `logret_*`) and compute $3 \times$ IQR bounds ($[Q_1 - 3 \times \text{IQR}, Q_3 + 3 \times \text{IQR}]$) strictly on the training split. We then clip returns in all three partitions using these training bounds. Across the partitions, 176 values were clipped in Train, 22 in Validation, and 102 in Test. Capped unscaled features are preserved in `data/processed/features_unscaled.csv` for exploratory data analysis.

Feature scaling. Asset price levels trade near \$2,000 to \$4,000, whereas daily returns fluctuate around 0.01. Unscaled features would cause gradient-based estimators and distance metrics to be dominated entirely by nominal price magnitudes. We fit `StandardScaler` from `scikit-learn` [17] on the training partition across 22 continuous numerical features, then apply this fitted transformation to the validation and test sets. Targets (`gold_5d_ahead`, `direction_5d`), the binary outlier flag (`is_outlier`), and cyclical trigonometric encodings (`*_sin`, `*_cos`) are explicitly excluded from scaling. Min-max scaling will also be explored for recurrent neural network architectures (LSTM) in Deliverable 2.

Distribution shift limitation. Because gold prices rose substantially during 2024–2026 compared with the 2010–2021 training baseline, the post-scaling mean of gold in the test set is 8.35, while the validation mean is 1.94. These non-zero out-of-sample means are consistent with a train-only scaler operating under a structural regime shift. In Deliverable 2, we will evaluate whether forecasting returns or differenced series mitigates this nominal price scale divergence.

5.5 Exploratory Data Analysis

We performed exploratory analysis on the engineered and unscaled dataset to inspect relationships, class balances, distributional fat tails, seasonality, and volatility regimes.

Return correlations. Figure 6 displays the correlation matrix of daily percentage returns across metals and external indicators. Gold and silver exhibit the strongest positive co-movement with a daily return correlation of 0.79. In contrast, the US Dollar Index (`dxy`) displays an inverse correlation of −0.37 with gold, consistent with gold tending to move opposite to the dollar. The 10-year Treasury yield (`us10y`, measured as a daily change in percentage points) exhibits a negative correlation of −0.24 with gold, indicating that gold returns tend to be lower on days when yields rise.

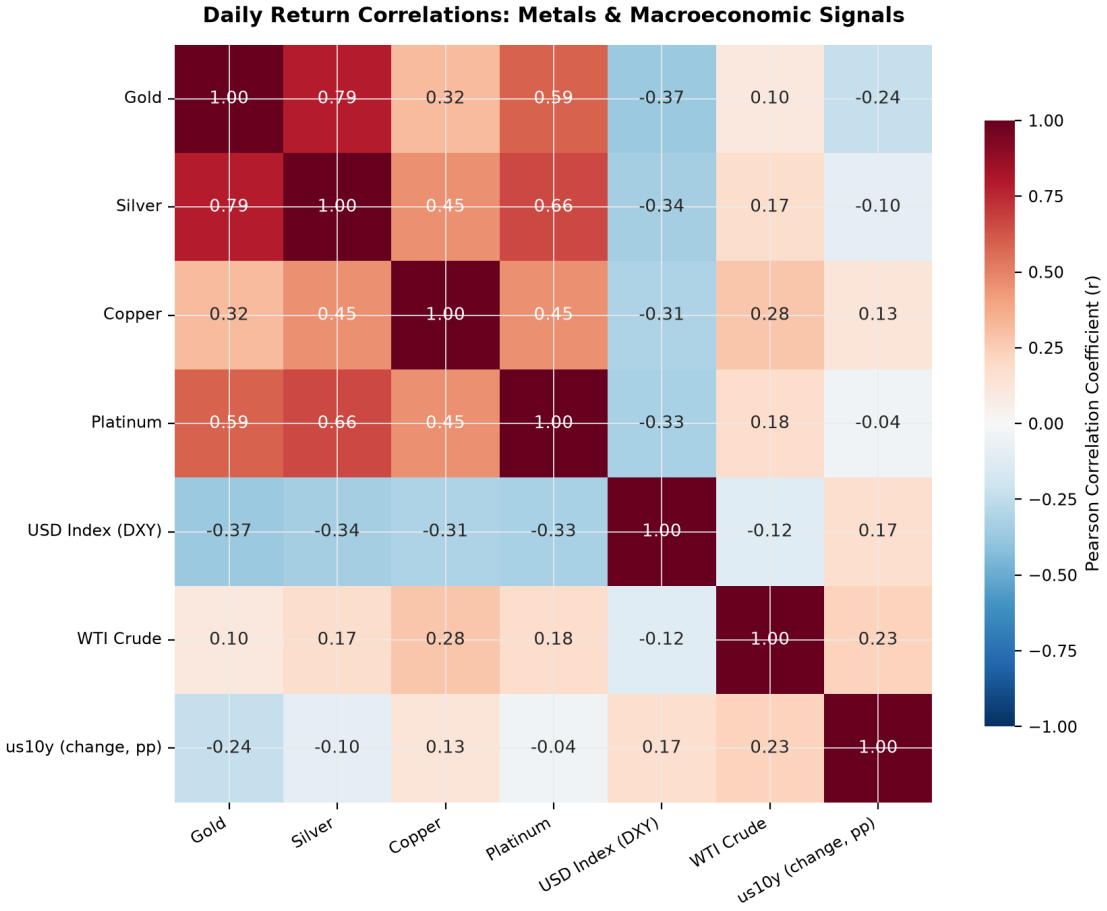


Figure 6: Correlation matrix of daily percentage returns and 10-year yield changes across metals and macroeconomic indicators.

Target class balance. Figure 7 plots target label frequencies across the chronological partitions. In the training split, 1,621 days were classified as Up (54.3%) and 1,367 as Down or Flat. In the validation split, 260 days were Up (51.8%) and 242 Down. In the out-of-sample test split, 426 days were Up (60.9%) and 274 Down. The target remains roughly balanced across train and validation, with a moderate upward shift in the test period reflecting recent market appreciation.

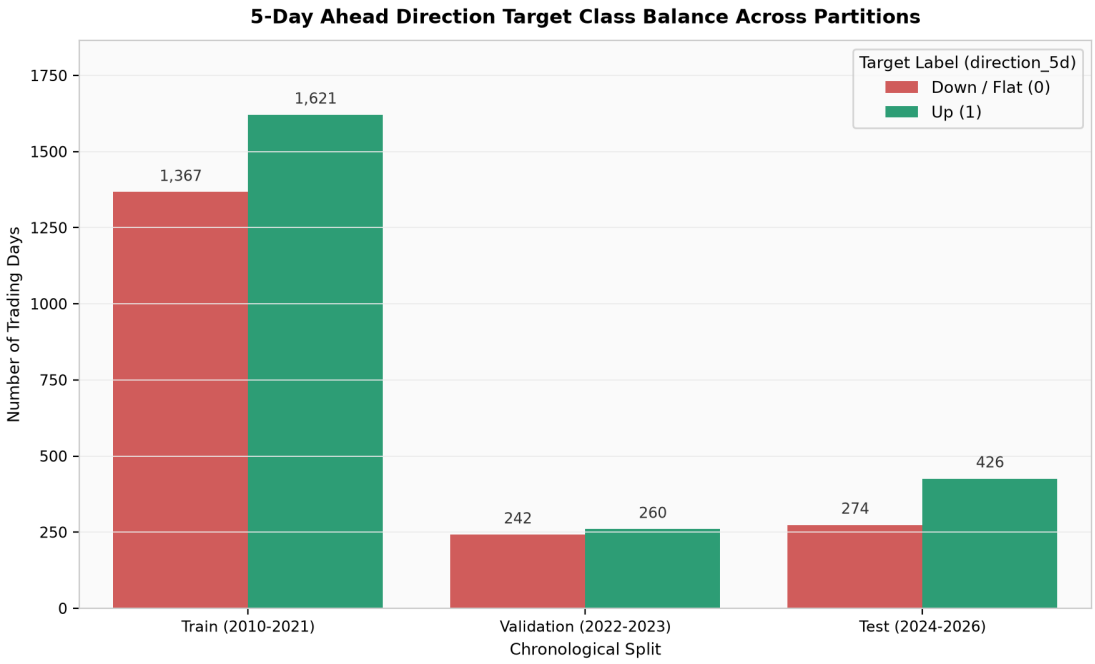


Figure 7: Class balance of the 5-day ahead directional target (`direction_5d`) across chronological partitions.

Return distributions and heavy tails. Figure 8 presents daily return histograms with kernel density estimates for gold, silver, and copper. In uncapped daily returns, all three metals exhibit severe excess kurtosis: +6.7 for gold, +15.6 for silver, and +13.8 for copper, accompanied by negative skewness (−0.60 for gold, −1.10 for silver, and −0.66 for copper). Applying the $3 \times$ IQR training-fitted cap reduces excess kurtosis to +1.7 for gold, +1.6 for silver, and +1.3 for copper, moderating extreme tail values while preserving the underlying distribution shape.

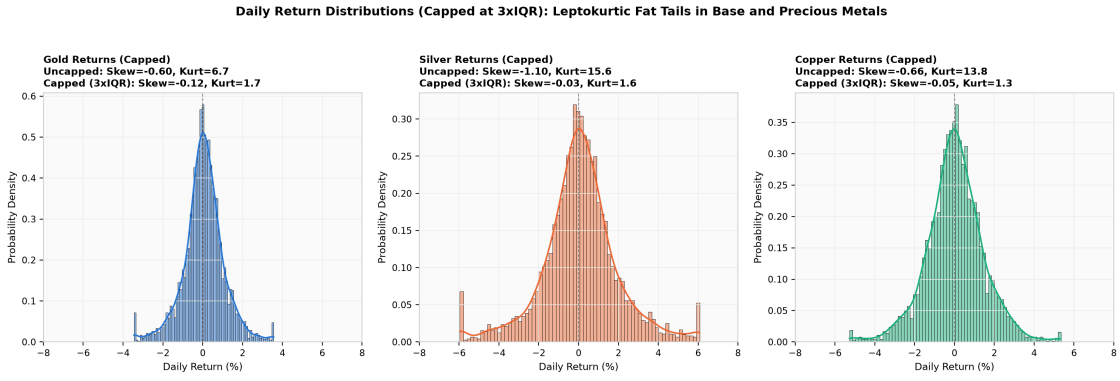


Figure 8: Daily return distributions for gold, silver, and copper under $3 \times$ IQR training-fitted capping.

Outlier profiles before and after capping. Figure 9 compares boxplots of raw returns against capped returns. Evaluating whisker outliers using the standard $1.5 \times$IQR rule yields 224 raw versus 226 capped outliers for gold (with 44 values resting at the cap), 281 raw versus 285 capped outliers

for silver (with 79 at the cap), and 200 raw versus 198 capped outliers for WTI crude (with 59 at the cap). Because capping enforces a $3 \times \text{IQR}$ threshold while boxplot whiskers extend to $1.5 \times \text{IQR}$, clipped observations still lie beyond the boxplot whiskers, but their extreme magnitude has been bounded.

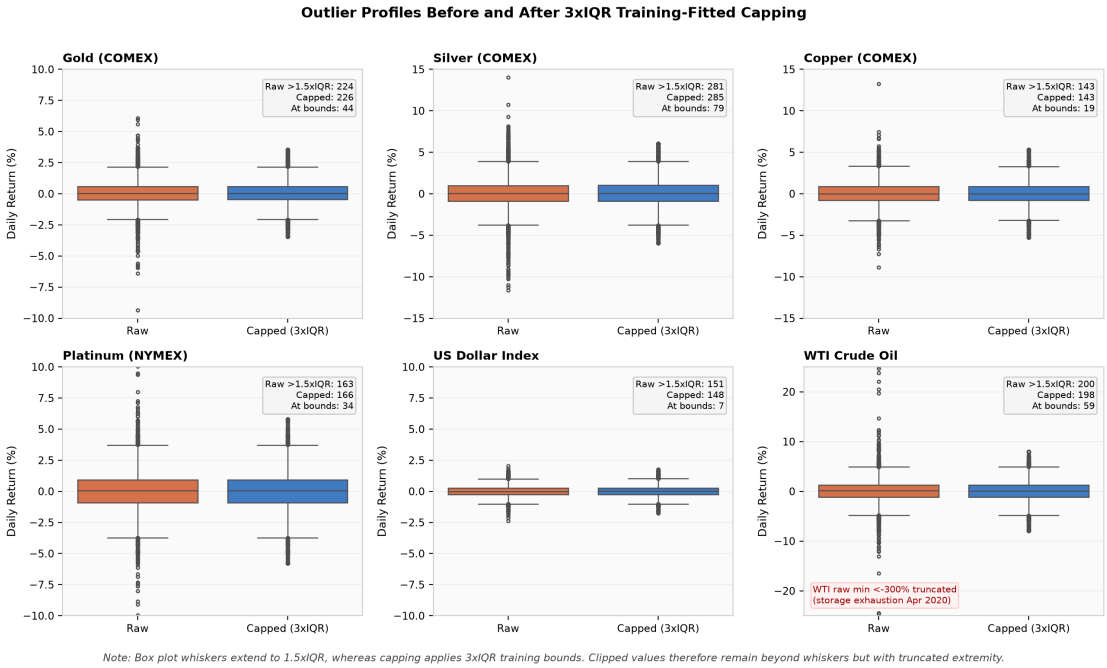


Figure 9: Return distributions across assets before and after $3 \times$ IQR training-fitted capping.

Seasonal decomposition. Figure 10 illustrates a multiplicative seasonal decomposition of monthly gold prices ($Y_t = T_t \times S_t \times I_t$, period 12) performed using `statsmodels` [19]. The estimated seasonal factor varies within a narrow band of 0.9797 to 1.0164, representing a maximum seasonal swing of about 3.67%. Over the sample period, the smoothed decomposition trend component rose by 262.1% (noting that the two-sided moving-average trend is undefined for the first and last six months), while raw gold closing prices rose by 269.8%. This indicates that, in our sample, the multi-year trend accounts for much more of gold's price movement than annual calendar seasonality.

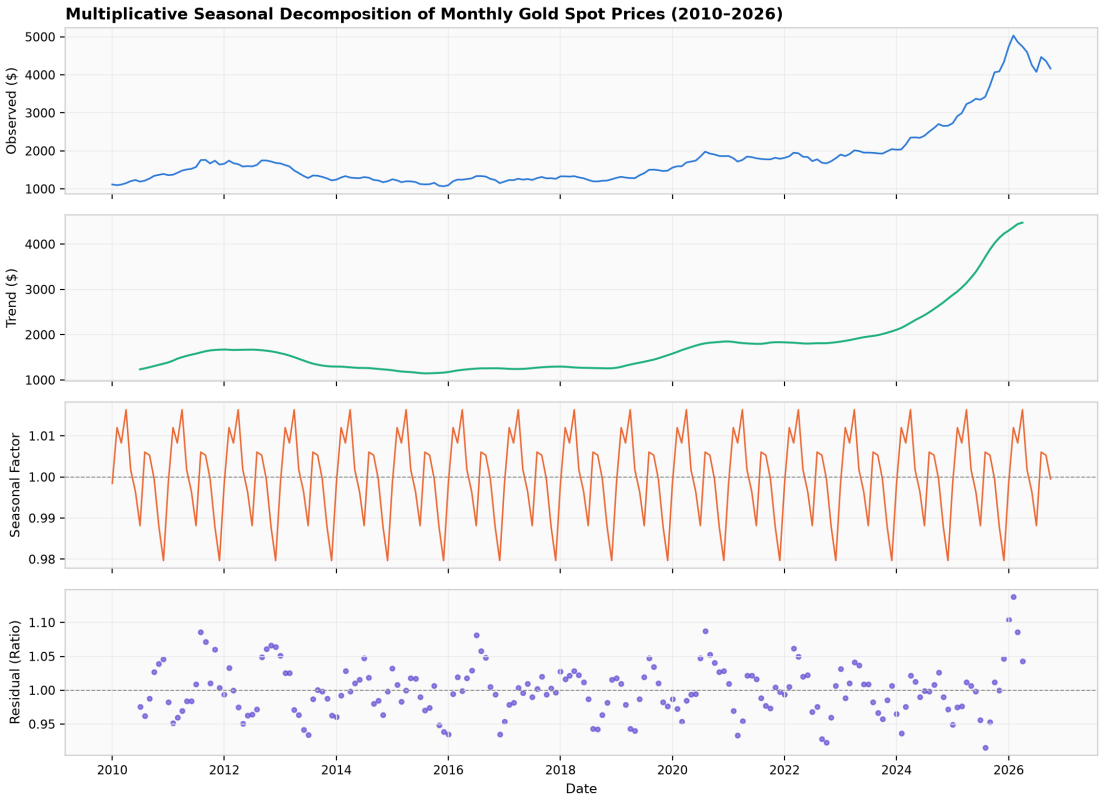


Figure 10: Multiplicative seasonal decomposition of monthly gold prices into trend, seasonal factor, and residuals.

Rolling volatility dynamics. Figure 11 plots the 30-day rolling standard deviation of daily gold returns over the 2010–2026 timeline. Over the sample, the median rolling daily volatility is 0.91%. The series reached a peak in our sample of 3.00% on 3 March 2026, alongside a COVID-period peak of 2.40% recorded on 24 March 2020. These peaks suggest that volatility varies across periods.

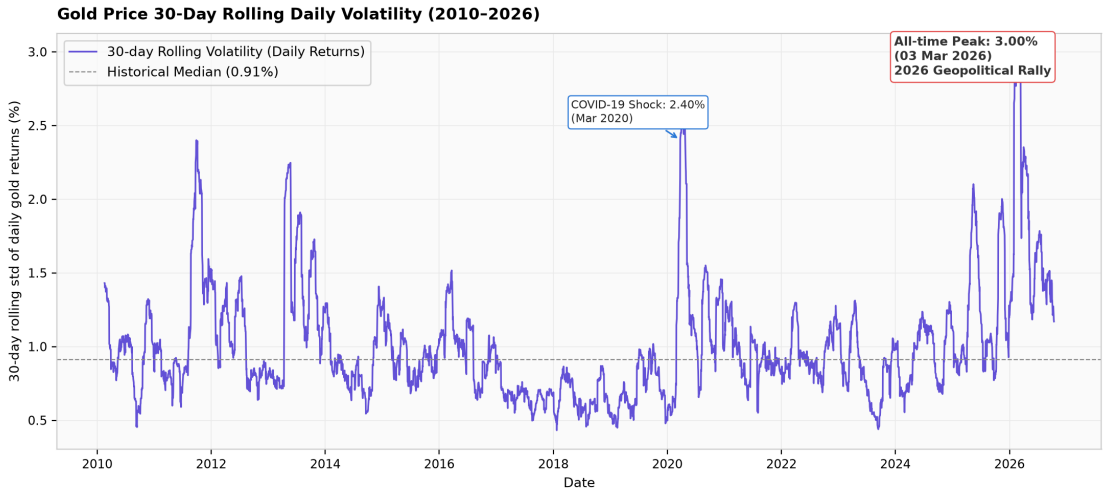


Figure 11: 30-day rolling standard deviation of daily gold returns (2010–2026) with annotated volatility peaks.

Gold sensitivity to the US Dollar. Figure 12 displays a scatter plot and ordinary least squares linear regression of daily gold returns against daily returns of the US Dollar Index. The estimated Pearson correlation is -0.369, with a regression slope of -0.857 and an R^2 of 0.136. While the negative slope indicates that gold returns tend to move opposite to the dollar, the modest R^2 indicates that currency movements explain only a small share of daily gold return variance, suggesting that other features are likely needed to explain gold returns.

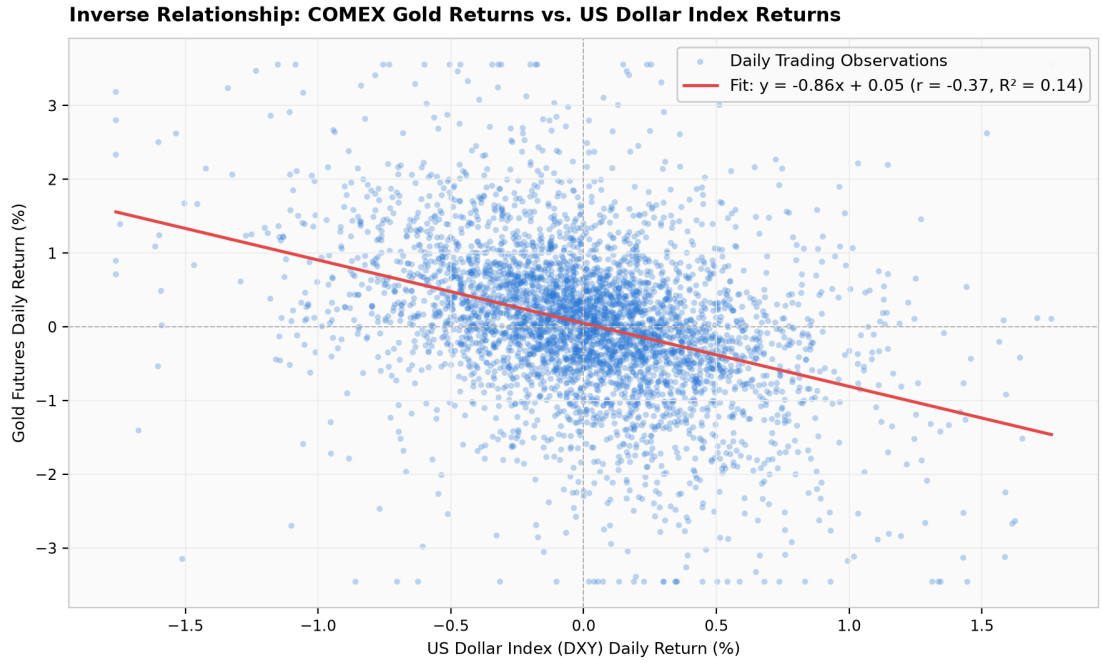


Figure 12: Scatter plot and linear regression fit of daily gold returns versus US Dollar Index returns.

6 Classical Models (Scikit-Learn)

6.1 Task and Framework Choice

Deliverable 2 treats forecasting as a binary classification problem. For each trading day the models predict `direction_5d`: whether the gold price will be higher five trading days later (1, "up") or not (0, "down or flat"). The input is one row of the features built in Section 5. Two feature sets are compared: **all** (27 columns) and **stationary** (14 columns), which removes the 13 price-level columns such as `gold`, `gold_lag5` and `gold_ma30` and keeps only returns, volatility, calendar terms and the outlier flag.

We use **scikit-learn** [18] for the classical models because the data is tabular and small (2,988 training rows), which is exactly where linear models and tree ensembles are strong. Scikit-learn gives every model the same `fit`/`predict_proba` interface, provides time-series cross-validation and grid search out of the box, and makes every run reproducible through a fixed random seed (42). Its histogram-based gradient boosting belongs to the same family as XGBoost [5], so no extra library is needed. The deep-learning model (Section 7) uses PyTorch.

6.2 Models

- **Majority baseline** (`DummyClassifier`): always predicts the most frequent class of the training set ("up"). Any useful model must beat it.
- **Logistic regression**: learns one weight per feature and turns the weighted sum into a probability with the logistic function. It can only draw a straight decision boundary, which makes it simple, stable and easy to interpret. The hyperparameter C is the inverse of the L2 regularisation strength.
- **Random forest** [4]: averages many decision trees, each trained on a bootstrap sample of the days and a random subset of the features. Averaging reduces the variance of single trees and captures non-linear interactions between features.
- **Histogram gradient boosting** [9]: builds shallow trees one after another, each correcting the errors of the previous ones. It is usually the strongest model on tabular data but also the easiest to overfit.

All models except the baseline use `class_weight="balanced"`, so that the slightly larger "up" class does not dominate training.

6.3 Hyperparameter Tuning

Each model is tuned with an exhaustive grid search (`GridSearchCV`) on the training split only, scored by AUC-ROC. The folds come from a walk-forward `TimeSeriesSplit` with five folds and a gap of five days (Section 8). Every grid is searched twice, once per feature set, giving 90 configurations in total; all of them are saved in `results/tuning/classical_tuning.csv`. Table 6 lists the values tested and the reason for each range.

Model	Values tested	Justification
Majority	none	Fixed reference point
Logistic regression	$C \in \{0.01, 0.1, 1, 10\}$	From strong to weak regularisation, on a log scale
Random forest	`n_estimators` $\in \{200, 500\}$; `max_depth` $\in \{3, 5, 8, \text{None}\}$; `min_samples_leaf` $\in \{1, 5, 20\}$	More trees reduce variance; depth and leaf size control how closely each tree can fit noise
Hist. gradient boosting	`learning_rate` $\in \{0.03, 0.1\}$; `max_depth` $\in \{3, 5\}$; `max_iter` $\in \{200, 500\}$; `l2_regularization` $\in \{0, 1\}$	Small learning rates and shallow trees are the standard defence against overfitting noisy data

Table 6: Hyperparameter grids for the classical models (16 to 24 configurations per tree model).

6.4 Final Choice

For every model the configuration with the best mean cross-validation AUC was kept for each feature set. The validation split (2022–2023) was then used once to choose between the two feature sets. Table 7 shows the result.

Model	Features	CV AUC (sd)	Val. AUC	Selected settings
Logistic regression	all	0.534 (0.058)	**0.530**	$C = 0.01$
	stationary	0.491 (0.037)	0.499	
Random forest	all	0.543 (0.050)	**0.538**	200 trees, depth 5, leaf 20
	stationary	0.491 (0.033)	0.505	
Hist. grad. boost.	all	0.547 (0.035)	**0.524**	rate 0.03, depth 3, 200 iter.
	stationary	0.499 (0.029)	0.485	
Majority	all	0.500 (0.000)	0.500	–

Table 7: Best configuration per model and feature set (CV: mean and standard deviation over 5 folds). Bold: selected.

Three observations guided the final choice:

1. **The signal is weak.** The best models reach a cross-validation AUC of only 0.53–0.55, three to five points above chance. Predicting the direction of a heavily traded market is hard, which is consistent with the efficient-market hypothesis [6].

2. **Strong regularisation wins.** Every model selected the most conservative settings available: the smallest C, the largest leaf size and the shallowest boosting trees. With a weak signal, simpler models generalise better.

3. **The full feature set was selected for every model**, because the stationary set scored at or below chance. The price-level columns therefore carry most of the usable information in the training period. As Section 8 shows, this choice has a cost: prices in the test period lie far outside the training range.

On the validation split the random forest is the best classical model (AUC 0.538). Gradient boosting had the best cross-validation score but dropped to 0.524 on validation, a sign of mild overfitting to the training period.

6.5 Feature Importance

Figure 13 shows the permutation importance [4] of the selected random forest: the drop in validation AUC when one feature is randomly shuffled. The most important features are the levels of platinum, copper, silver and oil and the lagged gold price. Even the most important feature changes AUC by less than 0.01, which again shows how little predictable structure the data contains. The reliance on price levels explains the behaviour of the tree models on the test set (Section 8).

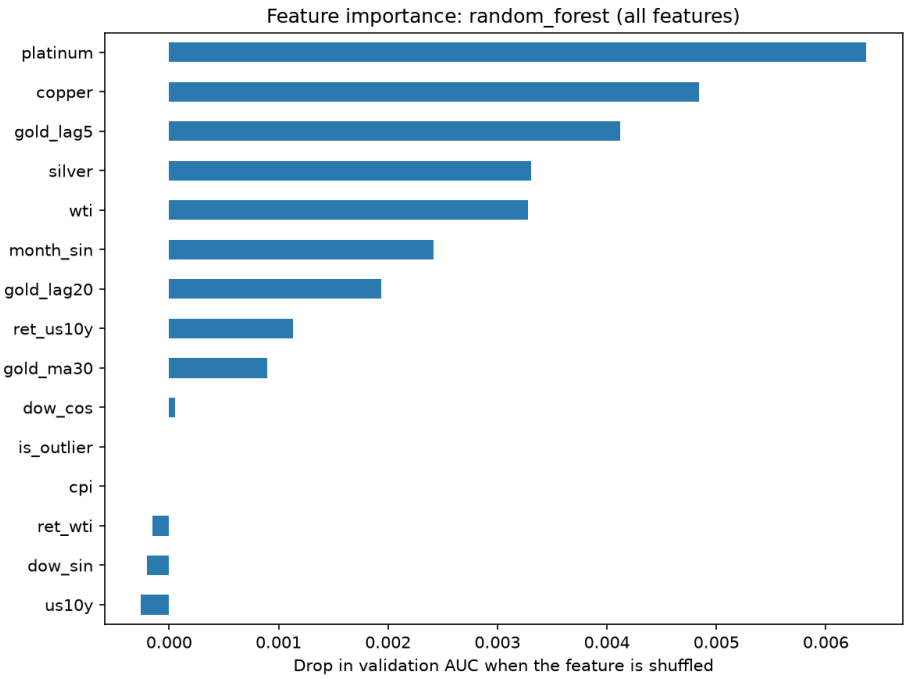


Figure 13: Permutation importance of the selected random forest on the validation split (top 15 features).

7 Deep Learning Model (PyTorch)

7.1 Framework and Motivation

PyTorch provides automatic differentiation, recurrent layers and an explicit training loop, which makes the sequence construction, loss calculation and checkpoint selection easy to inspect [16]. The long short-term memory (LSTM) network maintains a recurrent state over the input sequence [10]. It can therefore use how the features evolved over the last L trading days. The classical models receive one engineered row at a time, although that row already contains lagged features. This is a different representation of temporal information, not a guarantee that the LSTM will outperform those models.

Decimal metrics in this section and Section 9 are truncated towards zero at the displayed precision; counts and configuration values are exact.

7.2 Input Windows

The saved final configuration uses $L = 60$ trading days and $F = 14$ stationary features, giving an input tensor of shape $[\text{batch}, 60, 14]$. The label is `direction_5d` on the last day of the window: whether gold is higher after the following five trading days. Neither `direction_5d` nor the future price `gold_5d_ahead` is an input feature.

The split remains chronological: training covers 2010–2021, validation covers 2022–2023, and testing covers 2024–2026. A validation window can borrow the final $L - 1 = 59$ feature rows of training; a test window can borrow the corresponding rows of validation. These are observations from earlier dates, and their labels are not borrowed. This past-only context gives a prediction for every validation and test date without introducing future input features. The prediction file contains 2,929 training rows from 2,988 available training rows, followed by 502 validation rows and 701 test rows. Training loses its first 59 rows because no complete earlier window exists. Batches of training windows are shuffled, but neither split membership nor the chronological order within a window changes.

The `all` feature set contains 27 features. The `stationary` alternative removes the designated price-level features and retains 14 features. This choice reduces direct dependence on levels whose test-period distribution is far from the training distribution; Section 9 quantifies this for scaled gold. The name `stationary` describes the selected feature subset, rather than establishing statistical stationarity of every retained series. Both alternatives use the existing preprocessing fitted on training data.

7.3 Architecture

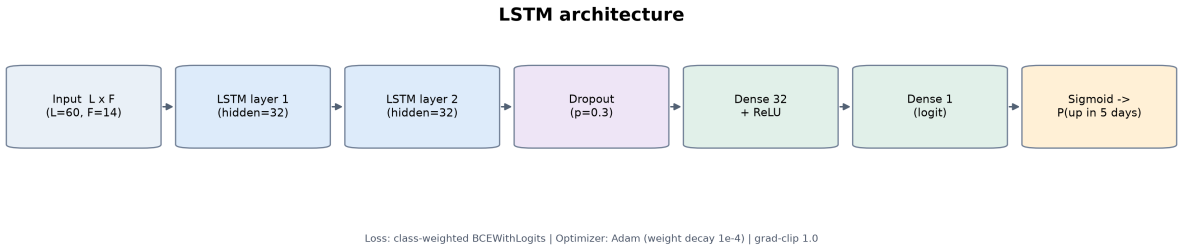


Figure 14: Architecture of the final LSTM. The sigmoid shown after the output is used for prediction; the model itself returns a logit.

Two stacked LSTM layers each have hidden size 32. Dropout with probability 0.3 is applied between the recurrent layers. The output at the last time step passes through a separate dropout layer with probability 0.3, a dense layer with 32 outputs, a ReLU, and a dense layer with one output. Dropout is intended to reduce overfitting by randomly suppressing activations during training [20]; it is disabled during evaluation. Instantiating the implemented model with the final configuration gives exactly 15,681 trainable parameters.

The final scalar is a logit, not a probability. The sigmoid is applied when generating probabilities and is incorporated into the loss calculation. This avoids placing a separate sigmoid inside the model before a loss that already handles it.

7.4 Loss and Optimizer

The fitting dataset purges the last five training windows because their labels look beyond the training boundary. Starting with the unpurged training predictions, dropping these five final dates leaves 2,924 labels: 1,345 down labels and 1,579 up labels. The class weight is therefore

$$\text{pos_weight} = \frac{\#\{y = 0\}}{\#\{y = 1\}} = \frac{1345}{1579} = 0.851804\ldots.$$

This value is passed as a float32 tensor to `BCEWithLogitsLoss`. It scales the positive-class contribution relative to the negative-class contribution. The combined sigmoid and binary cross-entropy calculation is numerically more stable than explicitly converting extreme logits to probabilities before evaluating the loss.

Adam [12] uses a learning rate of 0.001, a batch size of 32, and weight decay 10^{-4} to penalise large weights. Before each update, the total gradient norm is clipped at 1.0 to limit large recurrent-network gradients [15]. The Python, NumPy and PyTorch random seeds are all 42, and training runs on the CPU.

Unlike the training split, validation is not purged in the implemented loader. This preserves all validation predictions, but the final five validation labels can use prices from the test calendar period. Thus past-only input context prevents feature leakage, while the unpurged validation boundary remains a limitation of model selection.

7.5 Hyperparameter Tuning

Each configuration uses two recurrent layers, at most 100 epochs, and early-stopping patience of 10 epochs. Within a run, validation loss determines the checkpoint. Between runs, the highest validation AUC at that checkpoint determines the selected configuration; no test metric is used. Ties use lower validation loss when available, then grid order.

Features	Window	Hidden	Dropout	LR	Batch	Val. AUC
all	20	32	0.1	0.001	32	0.3879
stationary	20	32	0.1	0.001	32	0.5424
all	60	32	0.1	0.001	32	0.4448
stationary	60	32	0.1	0.001	32	0.5323
all	20	64	0.1	0.001	32	0.5001
stationary	20	64	0.1	0.001	32	0.4820
all	60	64	0.1	0.001	32	0.4941
stationary	60	64	0.1	0.001	32	0.5580
all	20	32	0.3	0.001	32	0.4365
stationary	20	32	0.3	0.001	32	0.5193
all	60	32	0.3	0.001	32	0.4585
stationary	**60**	**32**	**0.3**	**0.001**	**32**	**0.5934**
all	20	64	0.3	0.001	32	0.5149
stationary	20	64	0.3	0.001	32	0.4906
all	60	64	0.3	0.001	32	0.4780
stationary	60	64	0.3	0.001	32	0.5429
all	20	32	0.1	0.001	64	0.4277

Table 8: All evaluated LSTM configurations in saved grid order. The selected row is bold. Each AUC comes from a single seeded run.

Only 17 of the 64 planned configurations appear in the saved table: the grid was stopped early because of time. Every evaluated run uses learning rate 0.001, so the planned rate 0.0003 is untested. Both windows (20 and 60), feature sets (`all` and `stationary`), hidden sizes (32 and 64), and dropout probabilities (0.1 and 0.3) appear. However, 16 runs use batch size 32 and only one uses batch size 64; the combinations involving that larger batch are therefore mostly unevaluated.

Every configuration is a single run with seed 42. Differences of a few thousandths of AUC should be treated as potentially within run-to-run or sampling noise; that uncertainty was not measured. The selected stationary configuration has the largest recorded validation AUC, 0.5934, and is a defensible choice among the evaluated candidates. Its AUC is nevertheless an optimistic maximum over configurations assessed on one validation period, not an unbiased estimate of future performance.

Tuning used an earlier data download and only the final model was retrained on the frozen shared dataset; the training and validation rows are identical, so the selected configuration's validation result is unchanged, while only the test period differs.

7.6 Training Behaviour

Training ran for 15 epochs. The best checkpoint was epoch 5, with validation loss 0.635452; the next 10 epochs did not improve it. Early stopping restores the epoch-5 weights before saving predictions. The history evaluates both training and validation in evaluation mode with the same class-weighted loss. Its training metrics cover the purged fitting dataset, whereas the prediction CSV retains the additional final training windows.

Checkpoint	Epoch	Train loss	Val. loss	Train acc.	Val. acc.
Best	5	0.629206	0.635452	0.5526	0.5816
Last	15	0.494182	0.827428	0.7356	0.5139

Table 9: Evaluation-mode history at the best and last epochs. The last checkpoint is not used for the final predictions.

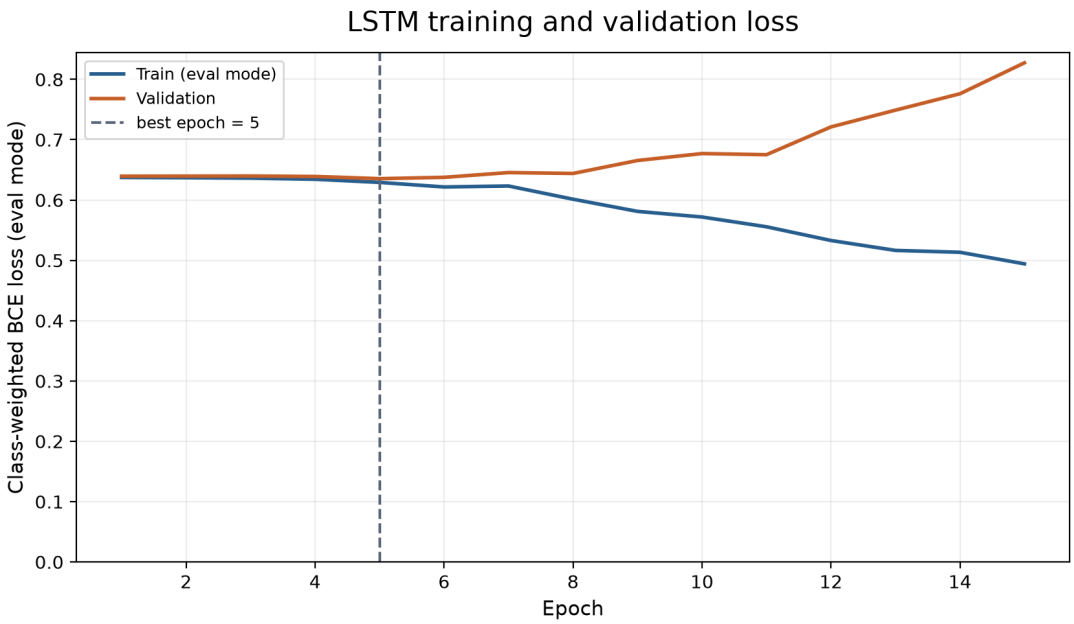


Figure 15: Class-weighted training and validation losses. The vertical line marks the minimum validation loss, which determines the restored checkpoint.

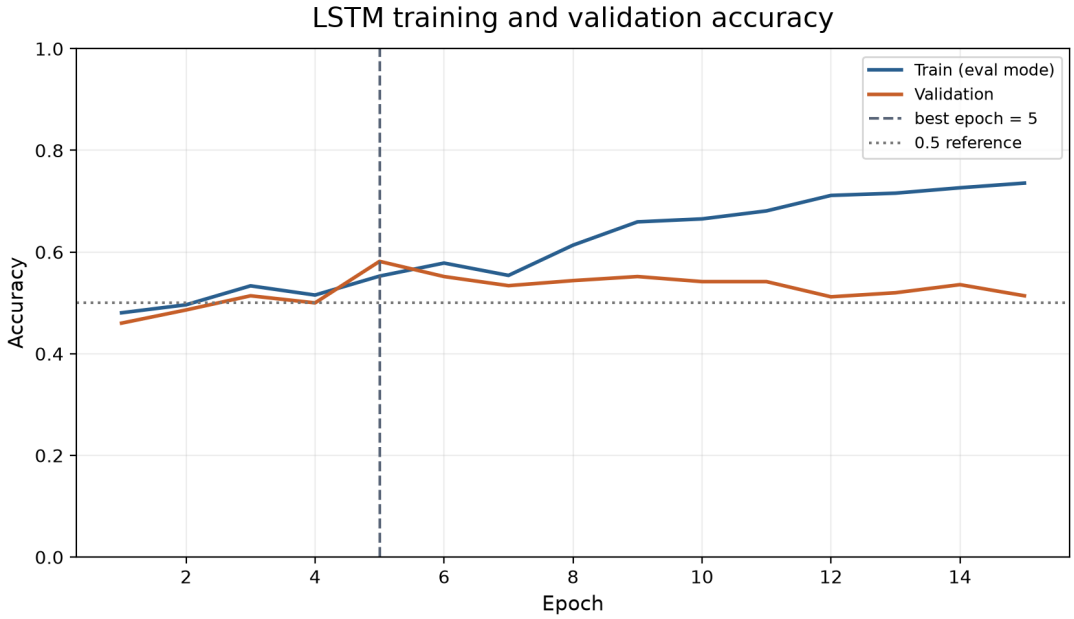


Figure 16: Training and validation accuracy in evaluation mode. The dotted horizontal line is the 0.5 reference, not the empirical majority baseline.

After the best epoch, training loss falls overall while validation loss rises, with fluctuations rather than a strictly monotonic trend. Training accuracy improves while validation accuracy deteriorates. These curves are consistent with overfitting to the fitting period and explain the need to restore the earlier checkpoint.

7.7 Result Summary

The final model's validation AUC is 0.5935. On the frozen test data, its accuracy is 0.5221, its predicted up-rate is 0.4593, and the true up-rate is 0.6091. It therefore falls below the always-up baseline accuracy of 0.6091 (Section 8), despite avoiding the complete class collapse discussed in Section 9.

On the shared test period (701 days) the final LSTM reaches a ROC-AUC of 0.5377, an F1 score of 0.5527, a precision of 0.6429 and a recall of 0.4848. The drop from the validation AUC of 0.5935 shows that the validation figure is optimistic, as it is the maximum over the evaluated configurations on a single period. A test AUC of 0.54 is only slightly above the 0.5 of a random ranking, so the model carries at most a weak directional signal. We therefore do not claim that the LSTM is useful for trading; it is mainly notable for not collapsing to a single class. These results do not establish a profitable trading strategy.

8 Evaluation

8.1 Validation Strategy

All models use the chronological split from Deliverable 1 (Table 5): training 2010–2021 (2,988 days, 71%), validation 2022–2023 (502 days, 12%) and test 2024–2026 (700 days, 17%). The data is never shuffled. A random train/test split or ordinary k-fold cross-validation would let a model learn from days that come *after* the days it is tested on. Because neighbouring days share most of their 5-day outcome window and markets move in long regimes, this leaks the future and overstates accuracy [2].

Inside the training split, hyperparameters are tuned with **walk-forward cross-validation**: `TimeSeriesSplit` with five folds, where each fold trains on all earlier data and validates on the next block. A **gap of five days** separates each training block from its validation block, and the last five rows of the training and validation splits are **purged** before fitting, because their labels look into the following period. This follows the purging practice recommended for financial data [13].

The validation split is used only for model selection: choosing the feature set of each classical model (Section 6) and early stopping of the LSTM (Section 7). The test split is used once, for the final results below.

8.2 Metric Selection

Accuracy alone is misleading here. Gold rose in 60.9% of the 5-day windows of the test period, so a model that always says "up" is right 60.9% of the time without learning anything. We therefore report four further metrics, treating "up" as the positive class (TP, FP and FN are true positives, false positives and false negatives):

- **Precision** $= \frac{TP}{TP+FP}$: when the model says prices will rise, how often it is right. For a buyer this is the reliability of a "buy now" signal; low precision means buying ahead of falls.
- **Recall** $= \frac{TP}{TP+FN}$: the share of rising weeks the model catches. Low recall means many missed chances to buy before a rise.
- **F1** $= 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision}+\text{Recall}}$: the harmonic mean, high only when both are high.
- **AUC-ROC** [7]: the probability that the model gives a randomly chosen rising week a higher score than a randomly chosen falling week. It does not depend on the 0.5 threshold or on the

share of rising weeks, so it stays comparable when the class balance shifts between periods, as it does here (54% up in training, 61% in test). It is our main metric for model selection.

8.3 Results

Table 10 compares all models on the test set (threshold 0.5). The full table for every split is in `results/tables/model_comparison.csv`.

Model	Accuracy	Precision	Recall	F1	AUC-ROC
Majority baseline	0.609	0.609	1.000	0.757	0.500
Logistic regression	0.498	**0.696**	0.311	0.430	0.569
Random forest	0.391	0.000	0.000	0.000	0.551
Hist. gradient boosting	0.391	0.000	0.000	0.000	**0.599**
LSTM	0.522	0.643	0.485	0.553	0.538

Table 10: Test-set results (2024–2026, 701 days, 60.9% up).

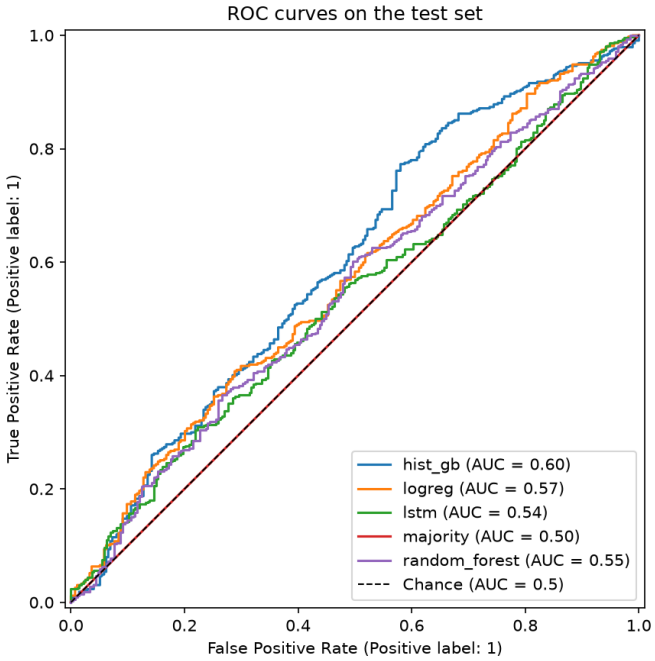


Figure 17: ROC curves on the test set. The dashed line is a random guess.

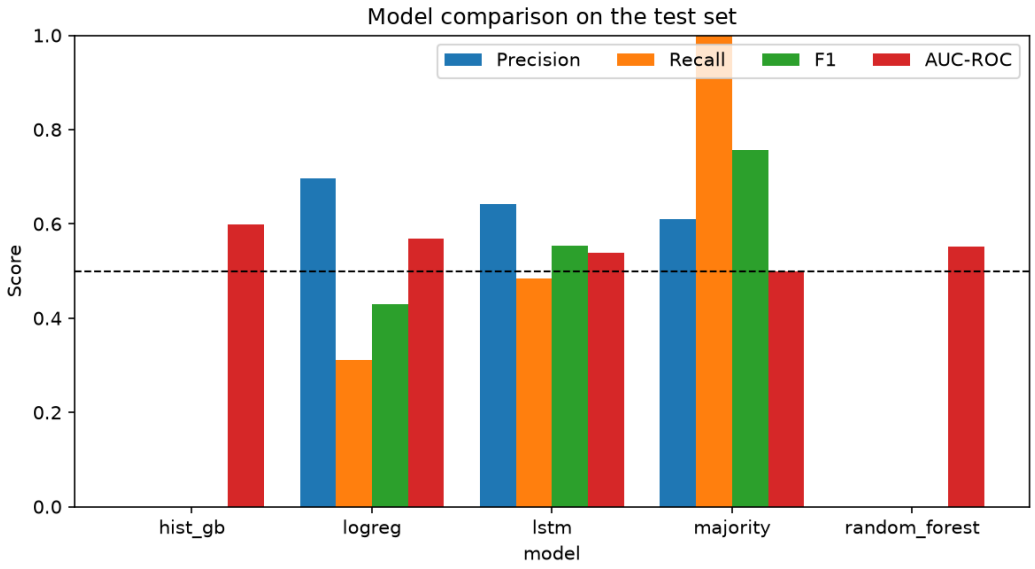


Figure 18: Precision, recall, F1 and AUC-ROC on the test set.

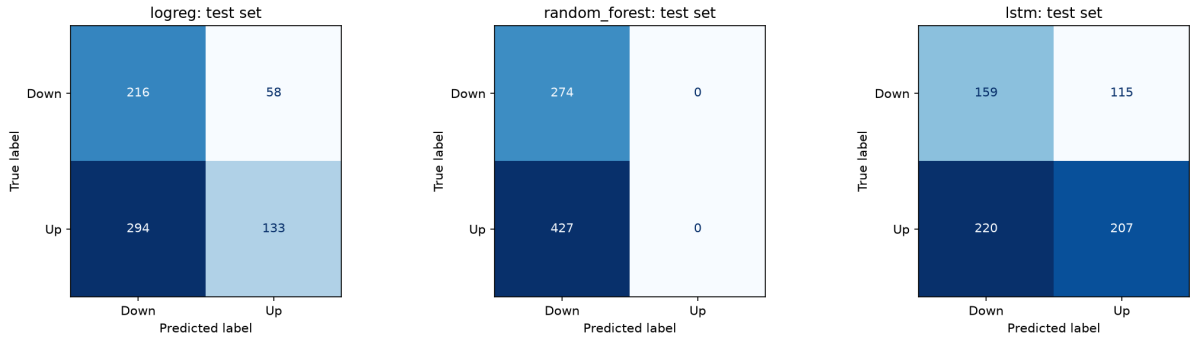


Figure 19: Test-set confusion matrices of logistic regression (left), random forest (middle) and the LSTM (right).

8.4 Discussion

- **The baseline has the best accuracy and F1**, purely because most test weeks went up. Its AUC of 0.500 shows that it has no ability to tell rising from falling weeks. This is exactly why accuracy and the F1 of the positive class cannot be used on their own.

- **Logistic regression gives the most useful signal.** When it predicts "up" it is right 69.6% of the time (133 of 191 calls), clearly above the 60.9% base rate, but it is cautious and catches only 31% of rising weeks. For a buyer, a rare but reliable "buy now" signal is more valuable than a frequent unreliable one.

- **The LSTM gives the most balanced predictions.** It has the best F1 of all learned models (0.553): it catches 48.5% of rising weeks (207 of 427) with 64.3% precision, and predicts "up" on 46% of test days instead of collapsing to one class. It uses only the stationary features (Section 7), so the rise in price levels does not push it to one side. Its ranking ability is weak,

however: the AUC falls from 0.594 on validation, the best of all models, to 0.538 on test, another sign that the relationship between features and direction changed between periods.

- **The tree models predict "down" on every one of the 701 test days** (recall 0, Figure 19), even though their AUC (0.551 and 0.599) shows that they still rank the days better than chance. All their test probabilities lie below 0.5 (at most 0.45). The cause is distribution shift: both models rely on price-level features (Figure 13), and gold traded far above its training range in 2024–2026 (the scaled gold level has a test mean of 8.35 standard deviations, Section 5). Trees cannot extrapolate, so every test day falls into the leaves for the highest prices seen in training, which were mostly followed by falls. Section 9 analyses this failure in detail.

- **Overall, the predictable signal is small**: the best AUC is 0.60, and no model beats the baseline's accuracy. This result is honest rather than disappointing: it shows that direction alone is a weak basis for procurement decisions, and it motivates the changes planned for Deliverable 3 (Section 13).

9 Error Analysis and Bias

There are no demographic subgroups in these market data. The relevant subgroups instead describe market conditions: the true direction class, volatility regime, outlier status, calendar year and named events. Class groups follow the true up/down label, outlier groups follow the saved `is_outlier` flag, and calendar groups follow the prediction date. Volatility groups use tercile cuts fitted once to unique training dates, then apply those same cuts to every model and split. Event windows cover March–April 2020 for COVID in training, calendar year 2022 for rate rises in validation, and calendar year 2026 for the surge in testing. These names identify periods, not estimated causal effects.

The five models, `majority`, `logreg`, `random_forest`, `hist_gb` and `lstm`, are evaluated on the same 502 validation dates and 701 test dates. The LSTM has fewer training predictions because its first complete window requires earlier context, so training comparisons use the rows actually available for each model. All results below come from the saved segment and bias tables. Decimal metrics follow the truncation convention in Section 7.

9.1 Where the Models Fail

Model	Accuracy	Recall up	Recall down
majority	0.6091	1.0000	0.0000
logreg	0.4978	0.3114	0.7883
random_forest	0.3908	0.0000	1.0000
hist_gb	0.3908	0.0000	1.0000
lstm	0.5221	0.4847	0.5802

Table 11: Test-split accuracy and class recall. Accuracy alone hides complete failure on one direction class.

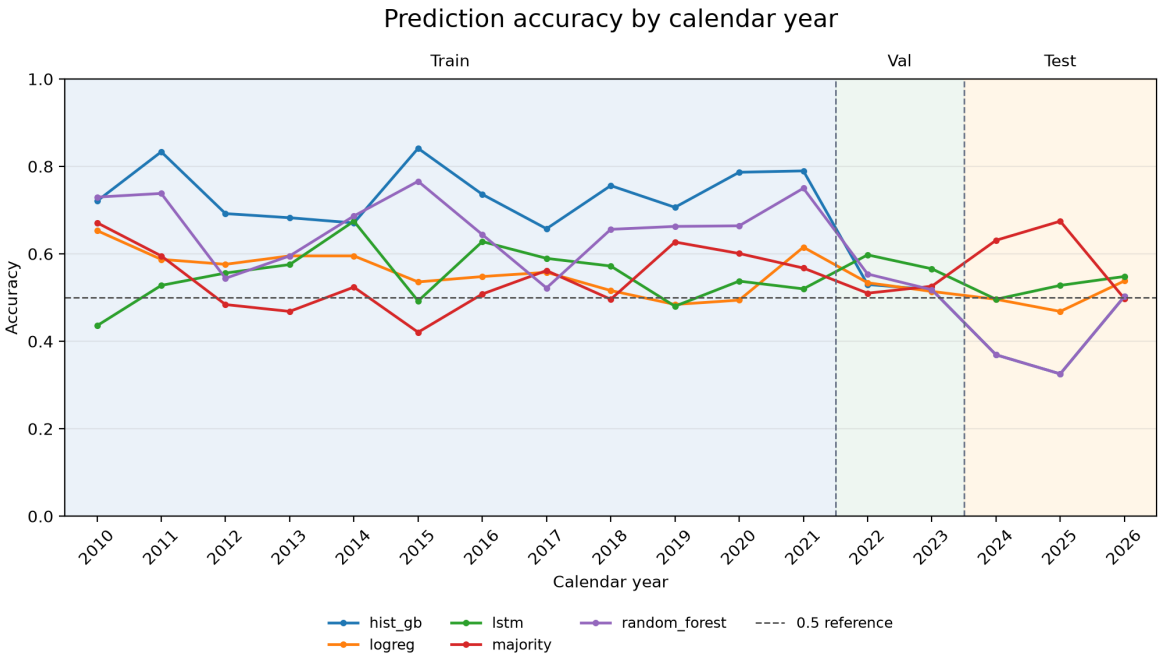


Figure 20: Accuracy by calendar year for all models. The split shading separates training, validation and testing; the horizontal dashed line marks 0.5. Training years are in-sample results.

Calendar years. The LSTM's test accuracy is 0.4960 in 2024 and 0.5277 in 2025, with 252 rows in each year. In 2026 it is 0.5482 on 197 rows, compared with 0.4974 for the majority model. Both tree models move from 0.3690 in 2024 and 0.3253 in 2025 to 0.5025 in 2026, while still predicting down on every test day. Their apparent improvement therefore reflects the changing class balance rather than recovered discrimination. Logistic regression reaches accuracy 0.5380 in 2026 but recalls only 0.0918 of the up cases in that year. The partial final year and the different class mixes limit direct comparisons across years.

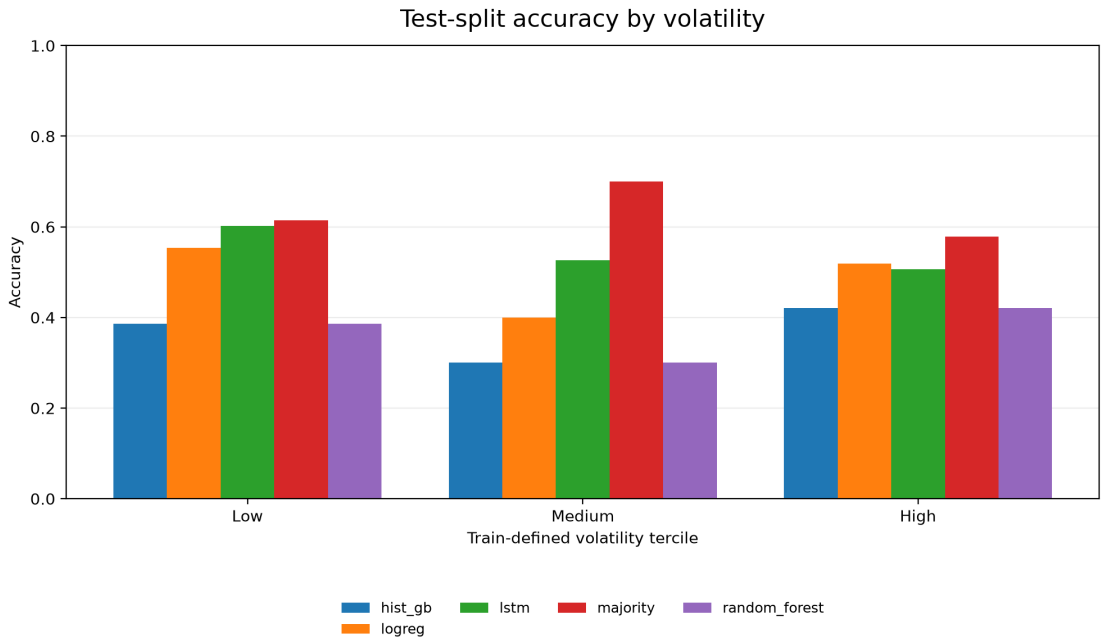


Figure 21: Test accuracy by volatility regime, using tercile thresholds fitted on training dates only. Test groups need not contain equal numbers of observations.

Volatility. The low-, medium- and high-volatility test groups contain 83, 150 and 468 rows. LSTM accuracy decreases across these groups from 0.6024 to 0.5266 to 0.5064. Logistic regression has accuracy 0.5192 in the high-volatility group, slightly above the LSTM there. The majority model has higher accuracy than every learned model within each volatility group. The low-volatility estimate is based on the smallest group and is correspondingly more uncertain; the row counts alone do not quantify that uncertainty.

Outlier days. There are 42 outlier rows and 659 normal rows in the test split. LSTM accuracy is 0.6190 on outliers and 0.5159 on normal days, compared with majority-model accuracies of 0.5952 and 0.6100. The apparent advantage on outliers is based on a small segment and is not evidence of a reliable advantage on rare events.

Named events. The COVID training window has only 43 rows per model. The LSTM has accuracy 0.5813, up recall 1.0000 and down recall 0.0526 in that window; `hist_gb` has accuracy 0.7674. These are in-sample results, not an independent crisis test. The 2022 rate-rise validation segment contains 251 rows, on which the LSTM's accuracy of 0.5976 is the highest among the saved models, but this period also contributed to model selection. The 2026 surge event contains the same 197 test rows as the calendar-year segment, with LSTM accuracy 0.5482; it supplies no additional independent evidence.

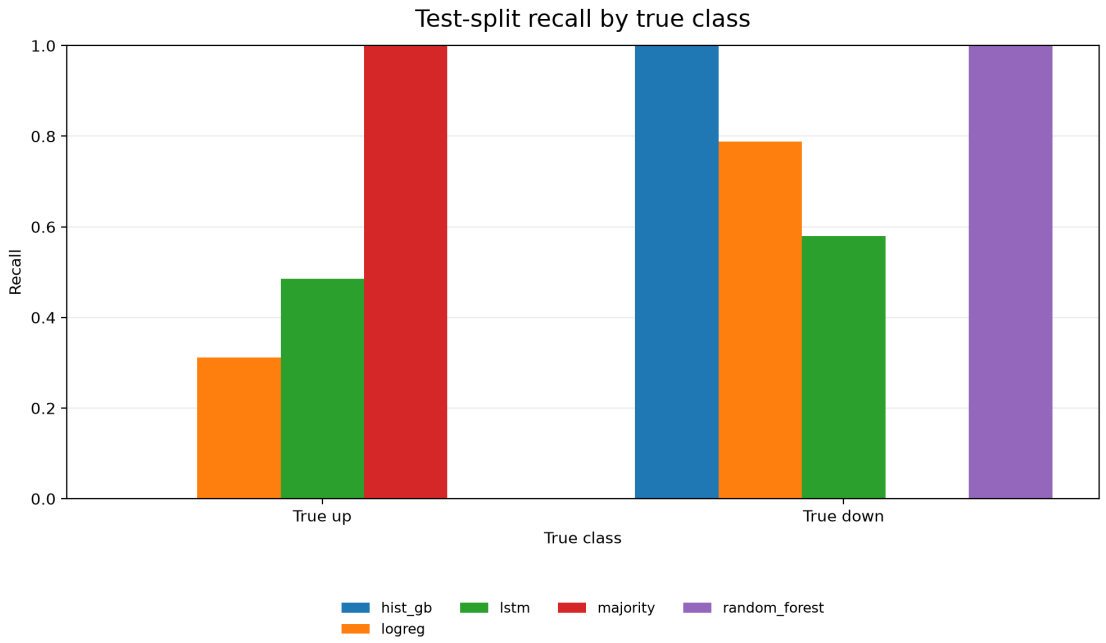


Figure 22: Test recall for up and down labels. A zero-height bar indicates zero recall for that class, rather than a missing model.

Consecutive five-day-ahead labels overlap in their future-price intervals, so the effective sample size is smaller than the row count. Small event and outlier segments are particularly noisy. Confidence intervals and statistical significance were not measured for these comparisons.

9.2 Bias Check

Here bias means a directional prediction imbalance rather than demographic unfairness. The gap is predicted up-rate minus true up-rate. The baseline predicts each model's own training-majority class on every split, rather than choosing the majority class from test labels. That training-majority class is up for every model in the saved data.

Model	True up rate	Pred. up rate	Gap	Majority accuracy	Model accuracy
majority	0.6091	1.0000	0.3908	0.6091	0.6091
logreg	0.6091	0.2724	-0.3366	0.6091	0.4978
random_forest	0.6091	0.0000	-0.6091	0.6091	0.3908
hist_gb	0.6091	0.0000	-0.6091	0.6091	0.3908
lstm	0.6091	0.4593	-0.1497	0.6091	0.5221

Table 12: Test-split directional bias and accuracy against the training-majority baseline. The gap is computed from the untruncated rates before display.

Class collapse. Both `random_forest` and `hist_gb` predict down on every test day: up recall is zero and down recall is one. The `majority` model always predicts up and has the opposite recall pattern. All three therefore meet the analysis flag "COLLAPSED to one class", although constant

prediction is intentional for the baseline. Logistic regression and the LSTM predict both classes, but both underpredict up relative to its observed frequency.

Distribution shift. The true up-rate changes from 54.25% on the classical models' training rows to 60.91% on test. The corresponding LSTM training rate is 53.97%, reflecting its shorter training prediction coverage. The 2024–2026 gold price-level distribution is also shifted far from training: the mean scaled test gold value is 8.3554 training standard deviations above the training mean. This shift is a plausible mechanism consistent with the collapse of the level-based tree models and the absence of complete collapse in the stationary-feature LSTM. It is not a causal explanation established by these experiments: the models differ in architecture, training and feature selection. Moreover, Hassan's classical tuning table shows that the best stationary-feature variant of each learned classical model did not beat its best all-feature variant on the recorded cross-validation score. Simply replacing all price levels is therefore a proposal for further testing, not an improvement demonstrated by these results.

Comparison with the baseline. None of the learned models beats always-up test accuracy, which is 0.6091; the majority model matches it by construction. The LSTM has the highest accuracy among the learned models at 0.5221 and the smallest absolute directional-rate gap among them at 0.1497. It is therefore the least biased learned model by this particular aggregate measure, while still failing to beat the baseline on accuracy. This does not establish general fairness or reliable performance in every regime: its high-volatility and normal-day results remain weak, and the more favourable outlier result has limited support.

Changes to evaluate next. The following are proposals, not measured improvements:

- Compare stationary and return-based feature sets across all model families, with past-only rolling or expanding re-scaling to reduce sensitivity to changing price levels.
- Use walk-forward refits as new data arrive and purge forward-looking labels consistently at validation boundaries.
- Tune decision thresholds or calibrate probabilities on validation data, then assess the fixed choices on untouched future periods.
- Repeat with multiple seeds, dependence-aware confidence intervals, and a longer or complete tuning grid.
- Assess AUC and balanced accuracy alongside ordinary accuracy, and evaluate transaction costs and trading utility separately before drawing investment conclusions.

Limitations. The LSTM evidence comes from a single seed, a partial grid, and one validation period used for both checkpoint selection and configuration selection. Its unpurged validation boundary can include forward-label prices from the test calendar period. Overlapping labels and small segments reduce the amount of independent evidence. Transaction costs and trading profitability were not evaluated. These predictive comparisons are not financial advice.

10 Buy, Hold and Negotiate Recommendations

The models in Sections 6 and 7 predict only the direction of gold. A buyer, however, needs an answer to a different question: *is this supplier's price good, and should I buy now?* This section describes how ProcurePulse turns market data and the model output into one of three actions for a supplier's quote. The code is in `src/recommendation/price_range.py` and is used by the Quote Check page of the application (Section 11).

10.1 Design Choice: AI Model for Gold, Price Range for All Metals

All models in this project were trained and evaluated for gold only. Training and validating separate models for silver, copper and platinum would repeat the whole of Deliverable 2 three times, and the gold results suggest that the signal would be weak for them as well (Section 8). We therefore use two levels of recommendation:

- **Gold: AI model and price range.** The quote is compared with an expected five-day price range, and the model's predicted direction decides quotes that fall inside that range.
- **Silver, copper and platinum: price range only.** The quote is compared with the expected range alone. The application labels these results "Price range only" so the user knows that no AI model was used.

10.2 Expected Five-Day Price Range

The range answers the question: *how far does this metal usually move in five trading days?* For a metal with daily price P_t, the five-day change is

$$r_t = \frac{P_{t+5}}{P_t} - 1.$$

We compute r_t for every trading day in the training years (2010 to 31 December 2021) and take its 10th, 50th and 90th percentiles, q_{10}, q_{50} and q_{90}. With the latest market price P_{last}, the expected range is

$$\text{low} = P_{\text{last}}(1 + q_{10}), \qquad \text{mid} = P_{\text{last}}(1 + q_{50}), \qquad \text{high} = P_{\text{last}}(1 + q_{90}).$$

By construction, eight out of ten five-day changes in the training years fell between q_{10} and q_{90}. This is an empirical prediction interval built from historical changes rather than from a fitted forecasting model [11]. Only the training years are used, so the range never contains information from the validation or test periods, in the same way as the scaler in Deliverable 1. The application calls the middle value the *fair price* and the upper value the *walk-away price*.

Table 13 shows the percentiles for the four metals. Each is based on 3,013 five-day changes. Silver moves the most and gold the least, so the same percentage markup can be acceptable for silver but too high for gold.

Metal (unit)	q_{10} (low)	q_{50} (mid)	q_{90} (high)	Width (points)
Gold (USD/oz)	−2.5%	+0.2%	+2.7%	5.2
Silver (USD/oz)	−4.4%	+0.1%	+4.8%	9.2
Copper (USD/lb)	−3.5%	+0.1%	+3.7%	7.2
Platinum (USD/oz)	−3.8%	−0.1%	+3.9%	7.7

Table 13: Percentiles of past five-day price changes, 2010–2021 (3,013 changes per metal).

10.3 Decision Rules

Table 14 lists the rules. A quote above the expected high is always **Negotiate**, because the market rarely rises that far in five days. When a model is used, a quote below the expected low is always **Buy**, and inside the range the model decides: if it gives a probability of at least 0.5 that gold will rise, waiting is likely to cost more, so the action is **Buy**; otherwise waiting is likely to be cheaper, so the action is **Hold**. Without a model (silver, copper and platinum, or gold when the model files are missing), the fair price splits the range instead.

Action	AI model + price range (gold)	Price range only (silver, copper, platinum)	Meaning
Buy	quote $<$ low, or low $\leq$ quote $\leq$ high and $p_{\text{up}} \geq 0.5$	quote $\leq$ mid	buy now
Hold	low $\leq$ quote $\leq$ high and $p_{\text{up}} < 0.5$	mid $<$ quote $\leq$ high	wait
Negotiate	quote $>$ high	quote $>$ high	ask for less

Table 14: Decision rules. p_{up} is the selected model's probability that gold is higher in five trading days.

Hold therefore has two meanings, and the application explains which one applies. With a model it means "the model expects the price to fall, so waiting may be cheaper". With the price range only it means "the quote is inside the usual range but above the fair price, so wait or ask for a small discount". The default gold model is logistic regression, because it gave the most reliable "up" calls on the test years (precision 0.70, Table 10). The user can switch to the random forest, gradient boosting or the LSTM. Because the random forest and gradient boosting predicted "down" on every test day, they will usually lead to **Hold** inside the range.

10.4 Worked Example

With the built-in data, the last gold price is 4,153.60 USD/oz on 21 October 2026. The expected range is 4,048.39 (low), 4,162.43 (fair price) and 4,264.93 (walk-away price). Table 15 shows three quotes for 10 oz. The cost compared with the market is (quote $- P_{\text{last}}$) $\times$ quantity.

Quote (USD/oz)	Cost vs market (10 oz)	Action	Reason
4,000.00	$-1,536$ USD	Buy	below the expected low
4,200.00	+464 USD	Buy or Hold	inside the range: model decides
4,300.00	$+1,464$ USD	Negotiate	above the walk-away price

Table 15: Gold quotes checked against the expected range (built-in data, 21 October 2026).

For the 4,200 quote the result depends on the model: it is **Buy** if the selected model expects a rise and **Hold** if it expects a fall. Without a model the same quote would be **Hold**, because it is above the fair price of 4,162.43.

10.5 Fresh Data

The built-in data ends on 21 October 2026, so the market price it uses becomes older every day. The user can check a quote against newer prices in two ways: by uploading a CSV file of recent

prices, or by choosing *Latest prices*, which downloads about one year of prices when the quote is checked (Section 11). In both cases the new prices pass through the same cleaning and feature steps as the training data. The models themselves are not retrained, and the range percentiles still come from 2010–2021, so the comparison stays consistent with the evaluated models.

10.6 Limitations

- **Exchange prices, not supplier prices.** The market price is the COMEX futures price. Real supplier quotes include premiums, taxes and delivery costs, so a fair supplier quote may sit slightly above the range.
- **Fixed width.** The range uses the same percentiles in calm and volatile periods. It does not widen when the market becomes more volatile.
- **Weak model signal.** The best test AUC is 0.60, so the model only tips quotes inside the range; it should be read as a guide, not a guarantee.
- **Buyers only.** The rules are written for a business that buys metal. Advice for sellers is left for future work.

11 Application (Track A)

11.1 Why Track A and Streamlit

Track A is implemented as a local web application that connects the saved models to a supplier-quote decision. Streamlit [21] keeps the interface in Python, the same language as preprocessing and inference, and provides interactive controls without a separate front-end implementation. Once dependencies and saved artifacts are present, the application starts with one command. A hosted service is not required for this demonstration. Here "real-time inference" means computing a result when the user requests it; it does not mean a streaming market feed or a measured latency guarantee. Response time and concurrent-user capacity were not measured for this report.

11.2 Application Architecture

The entry point, `app/streamlit_app.py`, registers four pages using `st.Page` and top navigation: Quote Check, Market, Model Insights and About. The page scripts call inference and recommendation functions, which read the saved models and data. This is the application-facing part of the pipeline in Figure 28; the architecture image is not repeated here.

- **Quote Check** — `app/views/quote_check.py`: reads cleaned prices, feature metadata, the saved scaler, classical model bundles, the LSTM checkpoint/configuration and the test comparison table. User uploads and downloaded prices provide alternative in-memory inputs.
- **Market** — `app/views/market.py`: reads `data/processed/cleaned.csv` through `src.inference.features.load_builtin_prices()`.
- **Model Insights** — `app/views/model_insights.py`: reads saved prediction, comparison, tuning, history and bias/segment CSVs under `results/`, together with existing report figures.
- **About** — `app/views/about.py`: reads dataset dates, the training feature header, model comparison and bias tables, `README.md` for the team, and the saved pipeline figure.

The inference and recommendation entry points are:

- `src.inference.features`: `build_features()`.
- `src.inference.classical`: `predict_classical()`.
- `src.inference.lstm`: `predict_lstm()`.
- `src.inference.live`: `download_latest()`.
- `src.recommendation.price_range`: `check_quote()` calls `expected_range()` and `recommend()` to produce the action and its explanation.

The pages reuse Deliverable 1 transformations and Deliverable 2 weights; they do not train models or regenerate evaluation artifacts. Application access to `data/`, `models/`, `results/` and `report/figures/` is read-only. This is an artifact-preservation statement, not a claim that the whole process cannot write: reused collection and cleaning helpers contain file-logging setup and logging calls.

11.3 Real-Time Inference Flow

The user chooses the metal, data source, quoted price and quantity, then presses **Check quote**. For gold, logistic regression is the default model because it is the first entry in `MODEL_NAMES`. The other classical options are random forest and gradient boosting; the majority baseline is a results-page comparator, not a quote-check model option.

The cached `gold_probabilities()` helper calls `build_features()` on the chosen prices. This reuses `data_scripts.feature_engineering.build_features()`, the training return caps derived from `features_unscaled.csv`, and `data/processed/scaler.joblib`. The result follows the 27-column training feature schema, excluding the future-price and direction targets. The saved scaler transforms its designated numeric columns; binary and calendar features retain their prescribed representation. Temporary repeated price rows allow feature construction at the latest observed date; the future targets and the padding rows are then removed, rather than supplied as model inputs.

`predict_classical()` selects each saved bundle's feature columns from the final row and returns its `predict_proba` up-probability. `predict_lstm()` uses the saved configuration: a 60-trading-day window and 14 stationary features selected from the same frame. CPU inference runs with evaluation mode and gradients disabled, followed by a sigmoid. Classical models are cached by `load_model()` using `lru_cache`; the LSTM model and configuration have a module-level cache. Streamlit also caches data and computed probabilities in memory.

An LSTM probability is displayed only after PyTorch, the configuration and weights load successfully. Missing dependencies, weights or invalid input cause the probability to be omitted. There is a narrower interface limitation: because PyTorch is imported lazily, the LSTM selector option can remain visible even when its weights or PyTorch are unavailable. Selecting it then leaves no LSTM probability, so the recommendation uses the price range alone. The selector itself is not a complete availability check.

For gold, `check_quote()` combines the selected probability with a five-trading-day price range. `expected_range()` applies the 10th, 50th and 90th percentiles of historical five-day returns through 31 December 2021 to the latest price. A quote above the upper bound leads to **Negotiate**; one below the lower bound leads to **Buy**. Within the range, an up-probability of at least 50% leads to **Buy**, otherwise **Hold**. Without a model probability, the median range value replaces the

probability decision. The displayed result includes reasons, the market price, quote difference and cost comparison for the entered quantity. Silver, copper and platinum always use the historical range only.

11.4 Data Options on Quote Check

- **Built-in data.** `load_builtin_prices()` loads the frozen cleaned table. The inspected dataset runs from 4 January 2010 to 21 October 2026; the page derives its displayed dates from the file rather than treating them as today's prices.
- **Upload CSV.** Required columns are `date`, `gold`, `silver`, `copper`, `platinum`, `dxy`, `wti`, `us10y` and `cpi`. `read_upload()` checks column names and dates, sorts the rows, removes weekends and duplicates, and reuses the cleaning functions for missing values, impossible prices and outlier flags. At least 100 trading-day rows must remain after cleaning. Parsing, missing-column, date and insufficient-history errors are shown as messages. A downloadable template is built from the final 120 rows of the built-in prices.
- **Latest prices.** `download_latest(days=365)` requests about one year of prices. Metals, the dollar index, oil and the yield backup come from Yahoo Finance; CPI comes from the US Bureau of Labor Statistics. It forward-fills CPI, excludes the current day and applies the upload-cleaning path. Internet access is required, and download failures produce an error message. Returned prices are passed through an in-memory `StringIO` CSV, not saved as a dataset. The page caches this result for 3,600 seconds. The logging qualification above still applies, so "nothing is saved to disk" would be too broad a claim.

The current implementation loads built-in data before enabling the other sources, and uses its earlier prices as historical context for range estimation. Upload and Latest prices therefore do not remove the need for the local reference dataset. If classical weights are missing, the page reports that AI inference could not run and falls back to the price range for the handled error cases.

11.5 Screenshots

The following figure slots correspond to the four application pages. Their descriptions are verified against the page code and saved data. Where a screenshot file is unavailable, the slot explicitly states this; dataset dates are not asserted to be screenshot capture dates.

Quote Check. The page accepts the supplier quote and quantity, explains the recommendation, and displays the selected model's role alongside the expected price range.

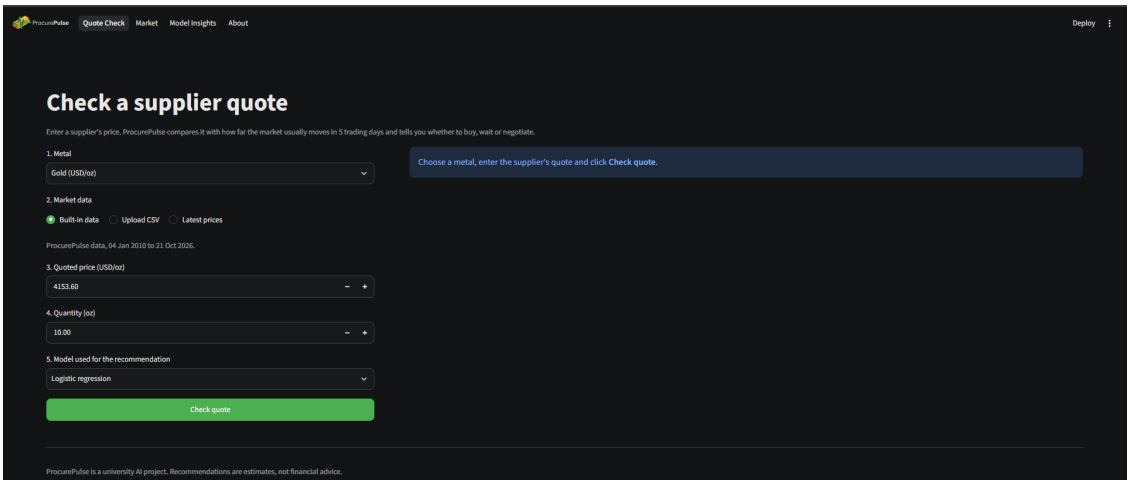


Figure 23: Quote Check: data-source and quote controls, recommendation and price-range explanation. The inspected built-in data ends on 21 October 2026.

Market. The page offers eight series, a date-range filter, prices with a 30-day average or daily returns, and summaries of price change, volatility and outlier days. An outlier flag belongs to the day, not to an individual series. Gold–dollar and gold–oil comparisons rescale each series over the selected interval.

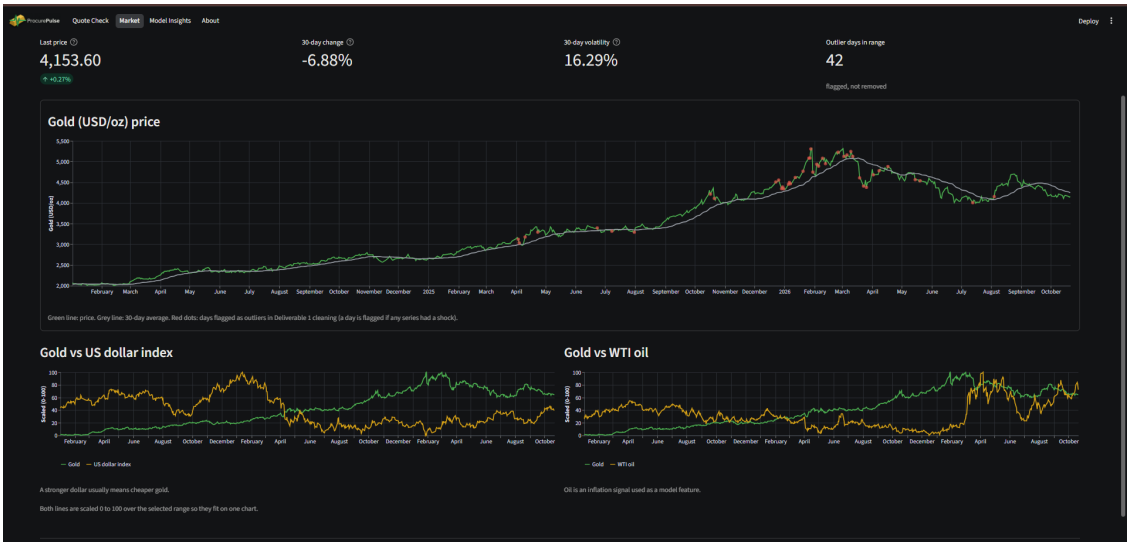


Figure 24: Market: historical series controls, trend and outlier views, and gold comparisons. The inspected dataset covers 4 January 2010–21 October 2026.

Model Insights. Its Overview, Training, Errors and Bias, and Sample predictions tabs expose saved metrics, learning curves, tuning tables, prediction imbalance and recent test probabilities. These are existing evaluation results, not recomputed model scores from the quote the user enters.

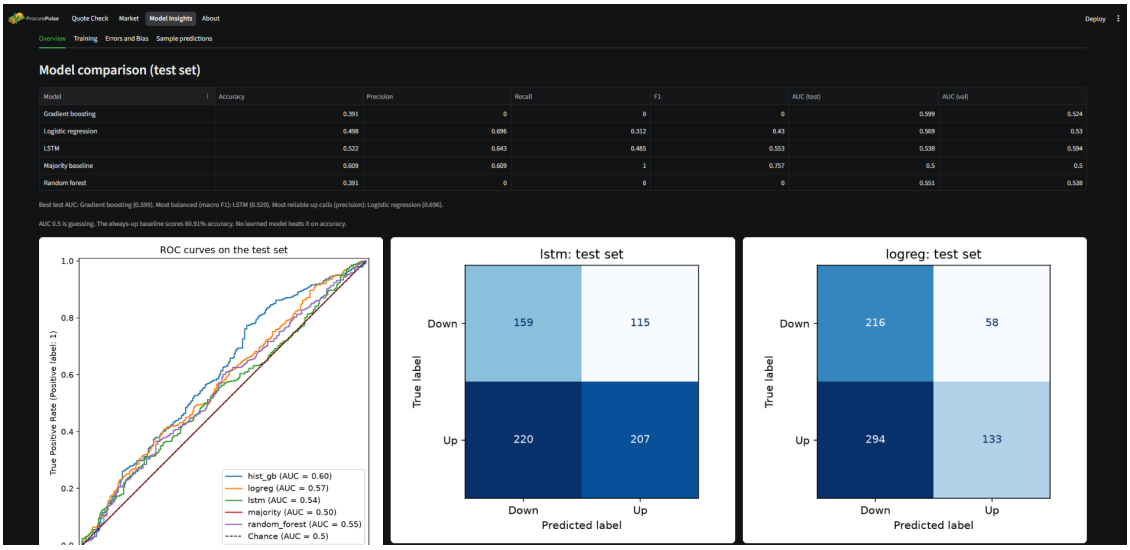


Figure 25: Model Insights: comparison and diagnostic views of the saved results. The prediction files contain 701 test days from 2 January 2024 to 14 October 2026.

About. The page describes the project, data options, limitations and team, with Terms of Service and Privacy Policy expanders. Data dates, feature count and best test AUC are derived from saved files; team information comes from the README. Missing supporting files produce availability messages.

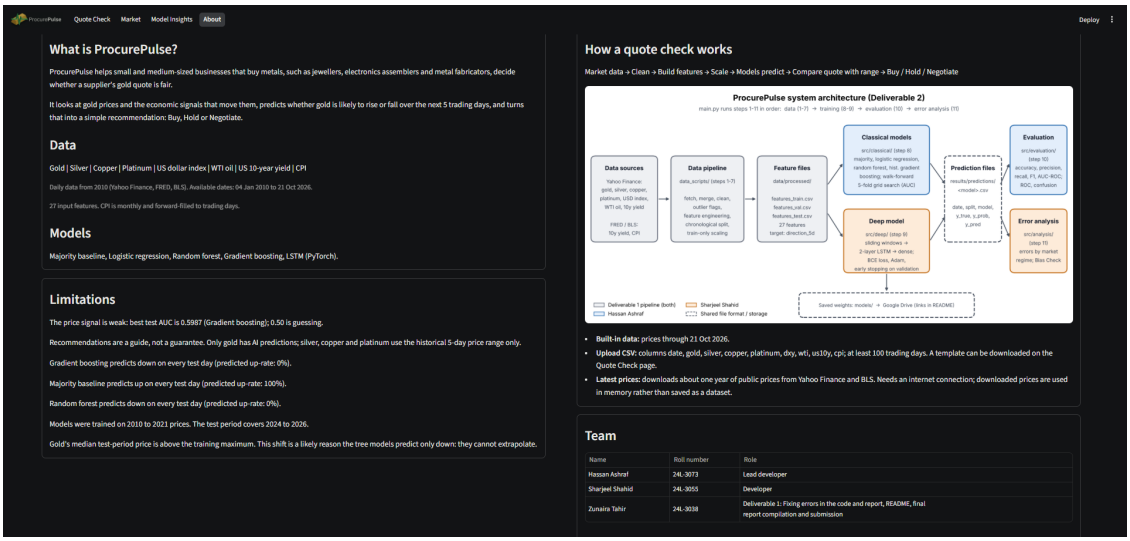


Figure 26: About: project context, data choices, limitations, team and policies. Its inspected built-in-data endpoint is 21 October 2026.

11.6 Sample Predictions

Figure 27 uses the last 120 test dates in each saved prediction file, 24 April–14 October 2026. The line is the predicted probability that gold rises over the next five trading days; the green and red dots show the realised binary outcomes. Classification uses $\hat{y} = \mathbf{1}[p \geq 0.5]$.

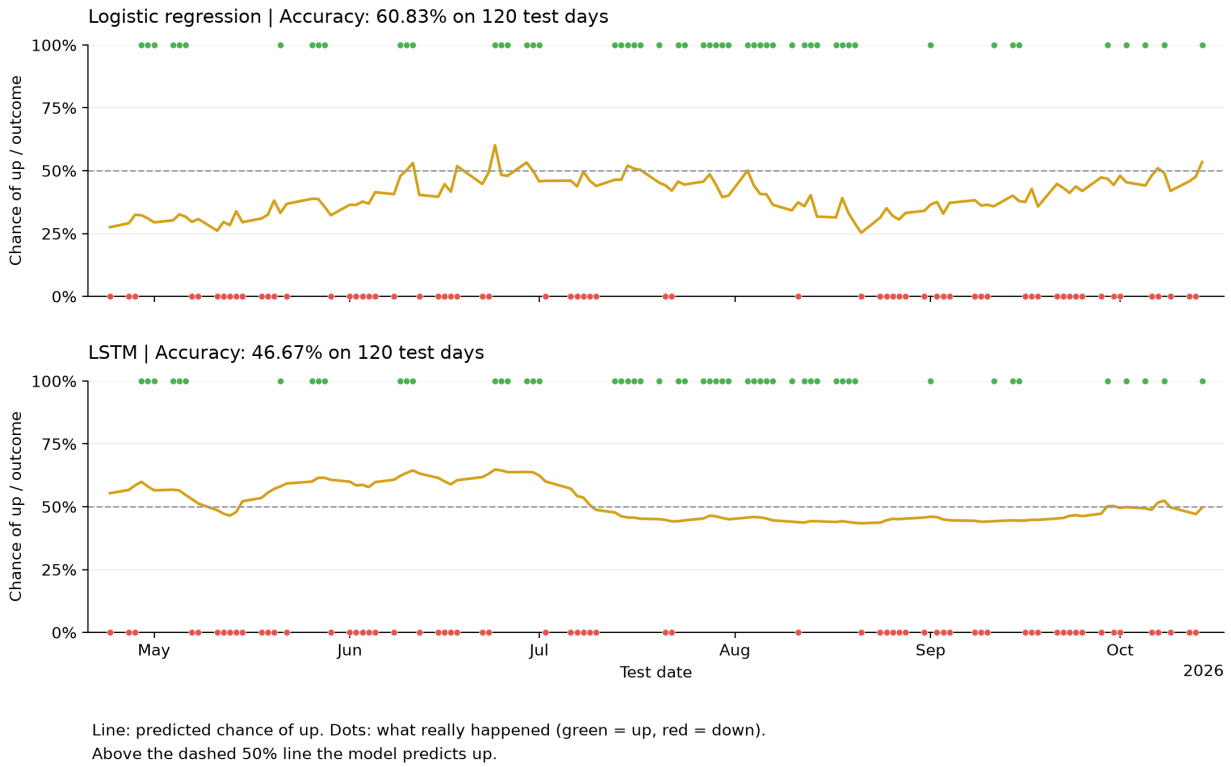


Figure 27: Saved logistic-regression and LSTM probabilities on the final 120 test days, 24 April–14 October 2026. Green dots denote actual up outcomes and red dots down or flat outcomes; the dashed line is the 50% decision threshold.

Logistic regression classifies 73 of these 120 outcomes correctly (60.83% accuracy), while the LSTM classifies 56 correctly (46.67%). Logistic regression is below 50% on 109 dates and above it on 11; the LSTM is below on 68 dates and above on 52. Neither has an exact threshold tie. Both therefore favour down predictions in this interval, especially logistic regression. These percentages are rounded to two decimal places, matching the figure.

The full-test AUC values in `model_comparison.csv` are 0.5690 for logistic regression and 0.5377 for the LSTM. They indicate weak discrimination above the 0.5 chance reference, not dependable trading signals. This recent window is a visual check of saved outputs, not an independent new test or evidence of profitability. Transaction costs and purchasing profit were not measured by this figure.

11.7 Running the Application and Limitations

From the repository root, with the project environment active, run:

```
python -m streamlit run app/streamlit_app.py
```

Install the dependencies in `requirements.txt`, which specifies `streamlit>=1.46`. The inspected environment has Streamlit 1.67.0. Follow the README's artifact layout. Keep classical `.joblib` bundles in `models/classical/`, and `lstm_best.pt` with `lstm_config.json` in `models/deep/`. Keep the cleaned prices, feature CSVs, unscaled features and `scaler.joblib` under `data/processed/`.

Retain saved results and figures for the diagnostic pages. Starting this interface does not require rerunning the README's training pipeline.

The predictive signal remains weak. In `bias_check.csv`, random forest and gradient boosting have a test predicted up-rate of zero: both predict down throughout the saved test period. The price-level distribution shift discussed in Section 9 is a plausible explanation, not a causal effect established by this interface. AI predictions are available only for gold; Latest prices depends on internet access and external providers. This is a local demonstration, not a hosted production service, and its recommendations are estimates rather than financial advice.

12 System Architecture and Contributions

12.1 System Architecture

Figure 28 shows how the parts of ProcurePulse fit together. The Deliverable 1 pipeline turns the raw downloads into chronological feature files. Two independent model families read these files: the scikit-learn models (Section 6) and the LSTM (Section 7). Both write their predictions in one agreed format, `results/predictions/<model>.csv`, which is the only connection between the two halves of the code. The evaluation (Section 8) and the error analysis (Section 9) read every prediction file, so a new model only has to produce this file to be included. The whole system runs with one command, `python main.py`, in eleven steps. Deliverable 3 adds an application layer on top of this pipeline: the Streamlit pages read the saved data, models and results and never retrain the models (Section 11).

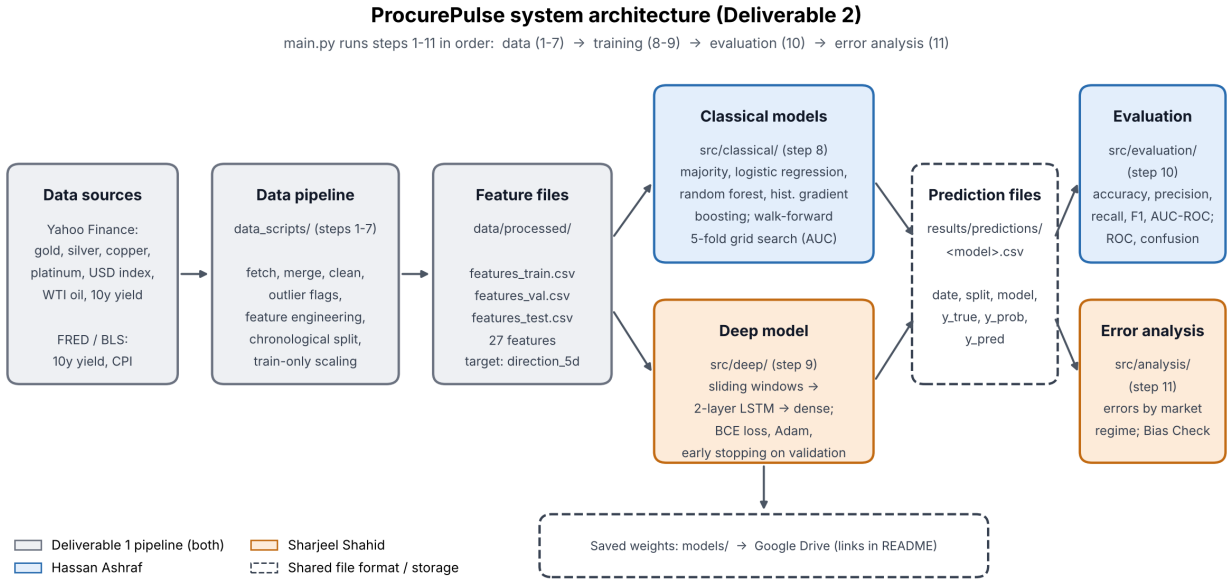


Figure 28: System architecture. Colours show who built each part in Deliverable 2.

12.2 Team

Table 16 shows what each member contributed to each deliverable. Zunaira Tahir was a member of the group for Deliverable 1 only.

Member	Deliverable 1	Deliverable 2	Deliverable 3
Hassan Ashraf 24L-3073 (lead)	Problem definition (Phase 1), data collection scripts, raw-data charts, report introduction, data collection and conclusion	Shared setup, scikit-learn models and tuning, evaluation module, `main.py` steps 8–11, report sections on classical models, evaluation, architecture and contributions, README	Price range and recommendation rule, application setup and theme, inference pipeline, Quote Check page, Latest prices download, report abstract, recommendations, introduction, conclusion and contributions, metrics chart fix, app screenshots, README, final report
Sharjeel Shahid 24L-3055	Data cleaning, feature engineering, exploratory data analysis (EDA), report sections on cleaning and features/EDA	PyTorch LSTM (dataset, model, training, tuning, curves), error analysis and bias check, report sections on the LSTM and the Bias Check	LSTM inference, Market page, Model Insights page, sample predictions figure, About page, report section on the application and its fixes
Zunaira Tahir 24L-3038	Fixing errors in the code and report, README, final report compilation and submission	–	–

Table 16: Team members and their contributions to each deliverable.

12.3 Contribution Table: Deliverable 3

Table 17 lists every Deliverable 3 module in the same way as for Deliverable 2: its files, the Git author and the number of commits, taken from the Git history of the `main` branch (`git log --no-merges`, Deliverable 3 commits only).

Task	Module	Files	Author	Commits
A1	Price range and recommendation rule	src/recommendation/price_range.py, tests/test_price_range.py	Hassan Ashraf	3
A2	Application setup, theme and logo	requirements.txt, .streamlit/config.toml, app/streamlit_app.py, app/assets/, report/main.tex	Hassan Ashraf	6
A3	Inference pipeline	src/inference/features.py, classical.py, tests/test_inference.py	Hassan Ashraf	3
A4	Quote Check page	app/views/quote_check.py	Hassan Ashraf	4
A5	Latest prices	src/inference/live.py, Quote Check data option, tests/test_live.py	Hassan Ashraf	3
A6	Report: abstract, recommendations, introduction, conclusion, contributions	00_abstract.tex, 13_recommendation.tex, 01_introduction.tex, 06_conclusion.tex, 11_contributions.tex	Hassan Ashraf	4
A7	Fixes, screenshots, README and final report	src/evaluation/plots.py and metrics chart, app/views/quote_check.py, report/figures/app/, README.md, 11_contributions.tex, ProcurePulse_Deliverable3.pdf	Hassan Ashraf	11
B1	LSTM inference	src/inference/lstm.py, tests/test_inference_lstm.py	Sharjeel Shahid	2
B2	Market page	app/views/market.py	Sharjeel Shahid	1
B3	Model Insights page	app/views/model_insights.py	Sharjeel Shahid	1
B4	Sample predictions and About page	src/analysis/sample_predictions.py, sample predictions figure, app/views/about.py	Sharjeel Shahid	3
B5	Report: application	Section 11, refs_b.bib	Sharjeel Shahid	2
B5	Application section fixes	Section 11, refs_b.bib, app/views/model_insights.py	Sharjeel Shahid	3
Total: Hassan Ashraf 34 commits, Sharjeel Shahid 12 commits				46

Table 17: Contribution Table for Deliverable 3 (from the Git history of `main`).

12.4 Contribution Table: Deliverable 2

Table 18 lists every Deliverable 2 module, its files, the Git author and the number of commits. The numbers are taken from the Git history of the `main` branch (`git log --no-merges`, Deliverable 2 commits only). Every module was developed on its own feature or fix branch and merged through a pull request with a merge commit.

Task	Module	Files	Author	Commits
A1	Shared setup	`requirements.txt`, `.gitignore`, `report/main.tex`	Hassan Ashraf	1
A2	Classical data and models	`src/classical/data.py`, `models.py`, `tests/test_classical.py`	Hassan Ashraf	3
A3	Classical training and tuning	`src/classical/train_classical.py`, classical predictions, tuning table, feature importance	Hassan Ashraf	3
A4	Evaluation	`src/evaluation/`, `tests/test_evaluation.py`, metrics, comparison table, charts (incl. final 5-model run)	Hassan Ashraf	8
A5	Pipeline integration	`main.py` (steps 8–11)	Hassan Ashraf	2
A6	Report: classical models, evaluation, conclusion	Sections 6, 8 and 13, `refs_a.bib` (incl. LSTM results)	Hassan Ashraf	5
A7	Architecture, contributions, final report	System diagram, `11_contributions.tex`, `ProcurePulse_Deliverable2.pdf`	Hassan Ashraf	4
A8	README	`README.md` (models, results, weights links)	Hassan Ashraf	2
B1	LSTM dataset	`src/deep/dataset.py`, `tests/test_deep.py`	Sharjeel Shahid	3
B2	LSTM model, loss and optimizer	`src/deep/model.py`, `tests/test_deep.py`	Sharjeel Shahid	2
B3	LSTM training	`src/deep/train_lstm.py`, LSTM predictions and history (incl. retraining on the shared dataset)	Sharjeel Shahid	4
B4	LSTM tuning and curves	`src/deep/tune_lstm.py`, `plot_training.py`, tuning table, curves, architecture diagram	Sharjeel Shahid	6
B5	Error analysis and bias check	`src/analysis/`, `tests/test_error_analysis.py`, segment and bias tables, figures	Sharjeel Shahid	4
B6	Report: deep model, error analysis	Sections 7 and 9, `refs_b.bib`	Sharjeel Shahid	4
Total: Hassan Ashraf 28 commits, Sharjeel Shahid 23 commits				51

Table 18: Contribution Table for Deliverable 2 (from the Git history of `main`).

13 Conclusion

Deliverable 1 established the foundations of ProcurePulse. We defined the problem: small and medium-sized businesses that buy metals face large and unpredictable price movements, and cannot easily judge whether a supplier's quote is fair. We built a reproducible pipeline that collects 16 years of daily data for four metals and four external signals, cleans it and turns it into model-ready features (Sections 3 to 5).

Deliverable 2 trained and evaluated the first models. We framed the task as predicting whether the gold price will be higher in five trading days, and compared a majority baseline, logistic re-

gression, a random forest and histogram gradient boosting (Section 6) with an LSTM (Section 7). All models were tuned with walk-forward cross-validation on the training years, selected on the validation years and tested once on 2024–2026 (Section 8).

The results are modest and informative. The best test AUC is 0.60, so the direction of gold is only slightly predictable from these features. Logistic regression produced the most reliable "up" signal, with 70% precision on its "up" calls, and the LSTM, which uses only stationary features, gave the most balanced predictions (F1 0.55). The tree models ranked days better than chance but predicted "down" on every test day, because they rely on price levels that moved far outside the training range. Accuracy alone would have hidden all of this, which is why we report precision, recall, F1 and AUC-ROC and analyse errors by market regime (Section 9).

Deliverable 3 turned these results into a tool a buyer can use. Every quote is compared with an expected five-day price range taken from the 10th, 50th and 90th percentiles of past price changes, which works for all four metals; for gold, the model's predicted direction decides quotes inside the range (Section 10). Because the model signal is weak, the range does most of the work and the model only tips the close cases. A local Streamlit application (Section 11) runs this inference in real time on built-in, uploaded or freshly downloaded prices, and explains each recommendation with the market price, the expected range and the model probabilities.

13.1 Future Work

- **Shift-resistant features**: replace raw price levels with relative measures, such as the price relative to its 30-day average, and retrain the models on a rolling window so they adapt to new price levels.
- **Calibrated decisions**: calibrate the predicted probabilities and choose the decision threshold on the validation split instead of using 0.5.
- **Forecasts with uncertainty**: Deliverable 1 planned regression forecasts with prediction intervals (ARIMA, Prophet and quantile models). Deliverable 3 used an empirical range from past price changes instead; a fitted forecasting model could give ranges that widen and narrow with market volatility.
- **AI models for the other metals**: train and evaluate direction models for silver, copper and platinum, so they no longer rely on the price range alone.
- **More realistic quotes**: add supplier premiums, local currencies and units, and a mode for businesses that sell metal.

References

[1] Ran Aroussi. yfinance: Download market data from yahoo! finance's api. `https://github.com/ranaroussi/yfinance`. Python library, accessed October 2026.

[2] Christoph Bergmeir and José M. Benítez. On the use of cross-validation for time series predictor evaluation. *Information Sciences*, 191:192–213, 2012.

[3] George E. P. Box, Gwilym M. Jenkins, Gregory C. Reinsel, and Greta M. Ljung. *Time Series Analysis: Forecasting and Control*. Wiley, 5th edition, 2015.

[4] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.

[5] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.

[6] Eugene F. Fama. Efficient capital markets: A review of theory and empirical work. *The Journal of Finance*, 25(2):383–417, 1970.

[7] Tom Fawcett. An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8):861–874, 2006.

[8] Federal Reserve Bank of St. Louis. FRED, federal reserve economic data. `https://fred.stlouisfed.org/`. Accessed October 2026.

[9] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001.

[10] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[11] Rob J. Hyndman and George Athanasopoulos. *Forecasting: Principles and Practice*. OTexts, Melbourne, Australia, 3rd edition, 2021.

[12] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.

[13] Marcos López de Prado. *Advances in Financial Machine Learning*. Wiley, 2018.

[14] Wes McKinney. Data structures for statistical computing in Python. In *Proceedings of the 9th Python in Science Conference*, pages 56–61, 2010.

[15] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. On the difficulty of training recurrent neural networks. In *Proceedings of the 30th International Conference on Machine Learning*, volume 28 of *Proceedings of Machine Learning Research*, 2013.

[16] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[17] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[18] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[19] Skipper Seabold and Josef Perktold. statsmodels: Econometric and statistical modeling with Python. In *Proceedings of the 9th Python in Science Conference*, pages 92–96, 2010.

[20] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56), 2014.

[21] Streamlit. https://streamlit.io. Installed version 1.67.0; accessed November 2026.

[22] Sean J. Taylor and Benjamin Letham. Forecasting at scale. *The American Statistician*, 72(1):37–45, 2018.

[23] U.S. Bureau of Labor Statistics. Public data API, version 2.0. `https://www.bls.gov/developers/`. Series CUSR0000SA0, accessed October 2026.